



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

**BorderGuessr: uno studio empirico
sugli algoritmi di ricerca per giochi a
somma zero con informazione perfetta
nell'ambito del learning by
gamification**

RELATORE

Prof. Fabio Palomba

Università degli Studi di Salerno

CANDIDATO

Gianfranco Vitiello

Matricola: 0512116703

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

*"L'ossessione batte il talento, lo batterà sempre.
Tu hai tutto il talento del mondo, ma hai anche
l'ossessione?"*

- Adam Sandler, Hustle

Abstract

I giochi a somma zero riscuotono un notevole interesse nella comunità scientifica per essenzialmente due motivi: nonostante possiedano spesso una struttura molto semplice, è molto complesso individuare strategie di gioco ottimali. Questo lavoro di tesi ha un duplice scopo:

- in primis, condurre un'analisi empirica delle principali soluzioni disponibili per questa tipologia di giochi su un caso specifico: **BorderGuessr**;
- in secundis, sfruttare questo gioco come strumento di contrasto all'analfabetismo geografico, sempre più diffuso nella popolazione, sviluppandone quindi una versione mobile, con l'obiettivo dichiarato di pubblicarla su Google Play Store.

Per quanto riguarda il primo traguardo prefissato, i risultati ottenuti confermano la bontà dell'intuizione avuta da Claude E. Shannon negli anni '50: è inutile esplorare l'intero albero di ricerca (nonché impossibile in molti casi). Infatti, anche con una piccola profondità di taglio e una buona funzione di valutazione, non solo si abbattano le tempistiche di computazione, ma inoltre, contro avversari non perfetti, un algoritmo subottimale può ottenere prestazioni migliori di uno ottimale in termini di qualità delle mosse. Infine, è stato possibile constatare che, senza potenza di calcolo adeguata, non ha senso concentrarsi sulla progettazione di soluzioni basate sul machine learning.

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	vi
1 Introduzione	1
1.1 Contesto generale e motivazioni	1
1.2 Obiettivi	2
1.3 Struttura della Tesi	3
2 Background e stato dell'arte	5
2.1 GIOCHI A SOMMA ZERO E ALGORITMI IN GRADO DI RISOLVERLI	5
2.1.1 Agenti intelligenti	5
2.1.2 Algoritmi di ricerca	6
2.1.3 Teoria dei giochi e ricerca con avversari	7
2.1.4 Contesto matematico	9
2.1.5 Algoritmo Greedy	9
2.1.6 Algoritmo Minimax puro	11
2.1.7 Algoritmo Alpha-Beta	13
2.1.8 Varianti Alpha-Beta	19
2.1.9 Algoritmo Minimax euristico	20
2.1.10 Reinforcement Learning	21

2.1.11	Monte Carlo tree search	21
2.1.12	Algoritmo AlphaZero	23
2.2	GEOGRAPHY ILLITERACY E LEARNING BY GAMIFICATION . .	25
2.2.1	Geography Illiteracy	25
2.2.2	Learning by Gamification	26
2.2.3	Stato dell'arte	27
3	Proposta di gioco: BorderGuessr	29
3.1	Genesi	29
3.2	Descrizione	30
3.3	Rilevanza per lo studio	31
4	Progettazione e implementazione	33
4.1	PROGETTAZIONE	33
4.1.1	Specifica PEAS	33
4.1.2	Formalizzazione del problema	34
4.1.3	Contesto e strumenti	35
4.1.4	Criteri per la scelta degli Stati e dei confini	36
4.2	IMPLEMENTAZIONE PER L'ANALISI EMPIRICA	36
4.2.1	Descrizione astratta e pseudo-codice	36
4.2.2	Descrizione tecnica	37
4.2.3	Esempio	38
4.2.4	Algoritmo Greedy	40
4.2.5	Algoritmo Minimax standard e Alpha-Beta pruning	41
4.2.6	Algoritmo Minimax euristico	44
4.2.7	Monte Carlo tree search	46
4.2.8	Algoritmo AlphaZero	48
4.3	IMPLEMENTAZIONE PER L'APP MOBILE	53
4.3.1	Ambiente di sviluppo ed architettura	53
4.3.2	Target di utenti e modalità di gioco	53
4.3.3	Navigational Path	54

5	Analisi empirica	56
5.1	Introduzione	56
5.2	Modalità conduzione analisi	57
5.3	Ambiente di esecuzione	59
5.4	Campionato 1	60
5.4.1	Partite	61
5.4.2	Classifica finale	64
5.5	Campionato 2	65
5.5.1	Partite	66
5.5.2	Classifica finale	68
5.6	Osservazioni	68
5.6.1	Campionato 1	68
5.6.2	Campionato 2	73
6	Conclusioni e sviluppi futuri	75
6.1	CONCLUSIONI	75
6.2	SVILUPPI FUTURI	76
	Bibliografia	79

Elenco delle figure

1.1	Roadmap	3
2.1	Esempio albero di gioco	10
2.2	Albero di gioco dopo che <i>MIN</i> ha scelto le azioni da eseguire per ogni possibile mossa di <i>MAX</i> (in rosso)	12
2.3	Albero di gioco dopo che <i>MAX</i> ha scelto l' azione da eseguire assu- mendo che <i>MIN</i> giochi in maniera perfetta	13
2.4	Esempio di scenario che consente la potatura in un albero di gioco . .	14
2.5	albero di partenza	15
2.6	albero di gioco aggiornato dopo aver esplorato il sottoalbero con radice <i>B</i>	15
2.7	albero di gioco con path da <i>B</i> alla radice aggiornato	16
2.8	albero di gioco con path da <i>C</i> alla radice aggiornato	17
2.9	albero di gioco con path da <i>D</i> alla radice aggiornato e conseguente potatura	18
4.1	Albero di gioco con Stato iniziale Lituania	39
4.2	Albero di gioco a partire da Lettonia	39
4.3	Albero di gioco a partire da Estonia	40
4.4	Esempio scelta effettuata da Greedy	41
4.5	Albero di gioco a partire da Lettonia	43

4.6	Albero di gioco a partire da Lituania	44
4.7	Albero di gioco a partire da Bielorussia	46
4.8	Albero MCTS a partire da Bielorussia	48
4.9	Tensore di input [1, 16]	50
4.10	Tensore di output [1, 6]	51
4.11	Ottimizzazione usando algoritmo Adam	52
4.12	Navigational path dell'app BorderGuessr	55
5.1	Contenuto di un report	59
5.2	report AlphaBetaPruning vs GreedySearch partendo dai Paesi Bassi .	70
5.3	report HeuristicMinimax vs GreedySearch partendo dai Paesi Bassi .	70
5.4	albero di gioco AlphaBetaPruning vs GreedySearch partendo dai Paesi Bassi	70
5.5	albero di gioco HeuristicMinimax vs GreedySearch partendo dai Paesi Bassi	71

Elenco delle tabelle

4.1	Dominio del gioco nell'esempio greedy	38
4.2	Dominio del gioco nell'esempio minimax	42
4.3	Elenco delle View dell'app BorderGuessr	55
5.1	I giornata campionato 1	61
5.2	II giornata campionato 1	61
5.3	III giornata campionato 1	62
5.4	IV giornata campionato 1	62
5.5	V giornata campionato 1	62
5.6	VI giornata campionato 1	63
5.7	VII giornata campionato 1	63
5.8	VIII giornata campionato 1	63
5.9	IX giornata campionato 1	64
5.10	X giornata campionato 1	64
5.11	Classifica finale campionato 1	64
5.12	I giornata campionato 2	66
5.13	II giornata campionato 2	66
5.14	III giornata campionato 2	66
5.15	IV giornata campionato 2	67
5.16	V giornata campionato 2	67

5.17	VI giornata campionato 2	67
5.18	Classifica finale campionato 2	68

CAPITOLO 1

Introduzione

1.1 Contesto generale e motivazioni

Durante l'anno accademico 2024/2025, corrispondente al mio terzo e ultimo anno di corso della Laurea Triennale in Informatica (L-31), presso l'Università degli Studi di Salerno, ho avuto la fortuna di inserire tra i miei esami a scelta quello di **Fondamenti di Intelligenza Artificiale**, tenuto dal professore Fabio Palomba, divenuto poi anche mio relatore. L'obiettivo del corso era quello di fornire i concetti fondamentali delle tecniche principali usate per creare agenti intelligenti. Di conseguenza era diviso in due parti:

- la prima faceva riferimento a quelli che sono gli algoritmi di ricerca, che si sono rivelati particolarmente stimolanti anche in prospettiva di un lavoro di tesi;
- la seconda, invece, introduceva al Machine Learning.

Fortunatamente, il corso non era semplicemente di natura teorica, bensì prevedeva la produzione di un progetto sulle tematiche affrontate, con annessa documentazione. Sin da subito, mi sono cimentato nella ricerca di qualcosa che potesse correlare la teoria dei giochi con quella che è una delle mie passioni più grandi: la geografia. Tematiche abbastanza scollegate tra loro, per le quali sono riuscito comunque a

trovare un punto d'incontro: **BorderGuessr**. BorderGuessr è un gioco basato su quelli che sono i confini politici attualmente esistenti sul pianeta Terra: si parte da un certo Stato (scelto casualmente o in base a qualche criterio) e i due sfidanti si alternano nell'individuare una nazione confinante con l'ultima appena detta (avendo cura di non menzionare Stati già detti). Vince chi lascia l'avversario senza mosse legali o lo induce a sbagliare. Ecco quindi che emerge la caratteristica tipica di un gioco a somma zero: *mors tua vita mea* (la vittoria del giocatore X porta ad una sconfitta del giocatore Y, suo avversario).

1.2 Obiettivi

Gli obiettivi portati avanti nel corso di questo lavoro di Tesi sono stati essenzialmente due:

- effettuare uno studio empirico delle soluzioni attualmente disponibili per risolvere giochi a somma zero con informazione perfetta, avendo cura di confutare quanto ci si aspettava conoscendo la teoria e quanto poi è effettivamente emerso nella pratica. Il tutto su un caso di studio ben definito, ovvero BorderGuessr cercando di capire quale potesse essere il miglior **trade-off** possibile tra qualità e tempo di esecuzione;
- provare in qualche modo a far appassionare alla geografia politica, o quanto meno consentire di uscire da una condizione di ignoranza (voluta o meno), persone di qualunque età. Il tutto, cercando di rendere l'apprendimento il meno pesante possibile, usando un approccio di tipo **learning by gamification**, traducibile sostanzialmente in "imparare giocando".

In sintesi, il lavoro di Tesi si propone di coniugare un'indagine rigorosa e metodica nel campo dei giochi a somma zero con la creazione di uno strumento ludico-educativo capace di stimolare l'interesse verso la geografia politica. È inoltre possibile sintetizzare il lavoro svolto con la seguente roadmap:

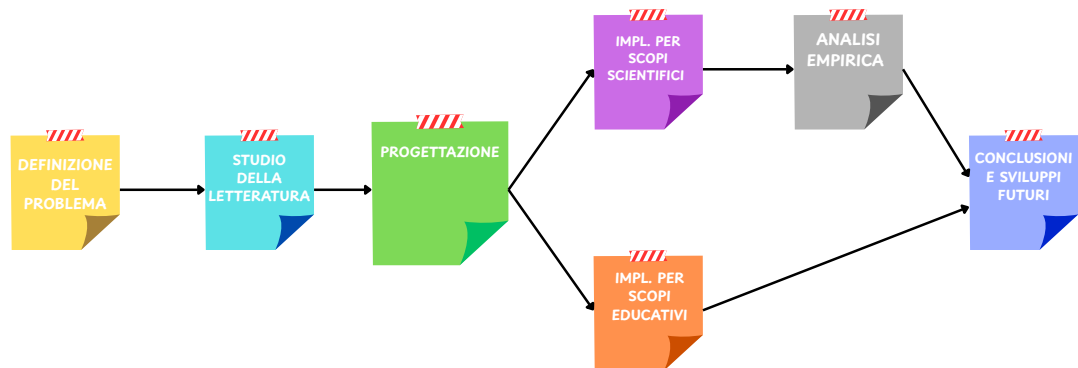


Figura 1.1: Roadmap

Maggiori informazioni su cosa e come sia stato fatto per ogni fase verranno fornite nella sezione successiva.

1.3 Struttura della Tesi

Tutti i capitoli, ad eccezione del terzo e del quinto, sono strutturati in due macro sezioni: la prima affronta gli aspetti scientifico-accademici, mentre la seconda si concentra su quelli ludico-divulgativi. Questa organizzazione permette di sviluppare in parallelo i due filoni discussi nella sezione precedente. Di seguito un breve riassunto del contenuto di ciascuno dei capitoli:

- **Capitolo 2:** discussione quello che è il contesto in cui il lavoro è stato svolto. Nella I parte vengono analizzati quelli che sono gli agenti intelligenti e come essi possano essere resi tali usando opportuni algoritmi. Dopodiché si passa alla discussione delle varie tipologie di soluzioni attualmente usate per i giochi a somma zero, partendo da quelle classiche, come la famiglia degli algoritmi Minimax, fino a quelle più moderne, facenti uso di tecniche come il Reinforcement Learning. Nella II parte, invece, viene introdotto brevemente il problema

della **geography illiteracy**, soffermandocisi su alcuni studi più o meno recenti e sottolineando le possibili conseguenze dell'avere questa tipologia di lacune;

- **Capitolo 3:** spiegazione dettagliata del funzionamento di BorderGuessr, con un po' di storia dietro la sua ideazione e del perché è stato scelto per condurre questa tipologia di studio;
- **Capitolo 4:** nella parte comune è stata svolta un'analisi tecnica e formalizzazione del gioco proposto, nonché definizione degli strumenti usati e dei criteri utilizzati classificare un'entità come Stato in maniera imparziale (che non necessariamente vuol dire oggettiva). Detto questo, la I parte si concentra sull'implementazione ai fini dell'analisi empirica di ciascun algoritmo mentre la II parte pone l'enfasi sull'architettura dell'implementazione scelta per gli aspetti ludico-divulgativi;
- **Capitolo 5:** è in questo capitolo che viene condotta effettivamente l'analisi empirica, simulando due campionati distinti dove ciascun algoritmo sfida tutti gli altri, in match sia come primo che come secondo giocatore, simulando game per tutte le nazioni disponibili. Seguono quindi delle osservazioni per ciascuno di essi che ne evidenziano punti di forza e debolezze;
- **Capitolo 6:** riflessioni finali sui risultati ottenuti ed eventuali proposte sia per migliorare lo studio che per rendere l'applicazione di carattere ludico più accattivante.

Sulla base di quanto appena detto, non resta altro che entrare nel vivo della trattazione con la discussione del contesto teorico in cui si inserisce questo lavoro.

CAPITOLO 2

Background e stato dell'arte

In questo capitolo verrà fornita una panoramica generale su quelli che sono gli agenti intelligenti e come essi sono in grado di prendere decisioni razionali. Seguirà poi un approfondimento sugli ambienti multi-agente e quali sono gli algoritmi più utilizzati oggi nei giochi a somma zero con informazione perfetta. Dopodiché, verrà introdotto il problema della "geography illiteracy" e come questa può essere contrastata utilizzando un approccio di carattere ludico, noto come "learning by gamification".

2.1 GIOCHI A SOMMA ZERO E ALGORITMI IN GRADO DI RISOLVERLI

2.1.1 Agenti intelligenti

Un agente è un sistema in grado di percepire l'ambiente esterno tramite sensori e di agire su di esso tramite degli attuatori. Questa definizione non è però sufficiente a descrivere un agente intelligente; infatti, per essere considerato tale, l'azione che esso intraprende deve risultare la migliore possibile rispetto ad una certa misura di prestazione. Di particolare interesse risultano essere gli agenti software, frutto del connubio tra un programma agente ed un'architettura (componentistica hardware). Come già anticipato, un agente agisce in un ambiente. Gli ambienti non sono tutti

uguali e, in base alle caratteristiche che posseggono, cambia anche l'approccio da utilizzare per individuare le azioni da eseguire. In particolare, Russell et al. [1, Capitolo 2], suggeriscono i seguenti criteri di classificazione degli ambienti:

- **grado di osservabilità:** in un ambiente completamente osservabile, è possibile misurare in qualsiasi momento gli aspetti che si reputano rilevanti per la scelta dell'azione da intraprendere. Al contrario, in un ambiente parzialmente osservabile non si ha accesso a tutti i dati di cui si ha bisogno (es. se l'agente è un'auto a guida autonoma non può conoscere le intenzioni degli altri veicoli);
- **numero di agenti:** distinguiamo ambienti a singolo agente e ambienti multi-agente. In quest'ultimo caso, gli agenti possono cooperare oppure competere;
- **grado di determinismo:** in un ambiente deterministico lo stato successivo dipende esclusivamente da quello corrente e/o dall'azione eseguita dall'agente. Al contrario, in un ambiente non deterministico lo stato successivo può dipendere da fattori esterni che possono verificarsi con una certa probabilità;
- **dipendenza tra le azioni:** se un'azione influenza anche quelle successive si parla di ambiente sequenziale; se invece ogni azione è scollegata da tutte le altre, si parla di ambiente episodico;
- **grado di mutabilità:** un ambiente si dice statico se non cambia mentre l'agente decide quale azione intraprendere, altrimenti risulta essere dinamico.

Ci sarebbero, in realtà, molti altri parametri da elencare ma, ai fini della trattazione, risultano essere sufficienti quelli esposti poc'anzi.

2.1.2 Algoritmi di ricerca

Per individuare l'azione migliore possibile da intraprendere per massimizzare la misura di prestazione (ovvero la funzione obiettivo che guida il comportamento dell'agente intelligente), gli agenti devono effettuare una vera e propria ricerca nello spazio delle azioni consentite. Ricerca che ovviamente segue un certo criterio, in base all'algoritmo scelto. Di base, sempre secondo Russell et al. [1, Capitoli 3, 4], è

possibile distinguere due tipologie di algoritmi di ricerca quando abbiamo a che fare con ambienti completamente osservabili e deterministici:

- **non informata**, in cui l'agente non dispone di informazioni utili che possano in qualche modo orientare la ricerca in un particolare sottoinsieme di azioni (es. DFS, BFS, Dijkstra, ...);
- **informata**, dove l'agente è guidato da euristiche. Un'euristica è una sorta di "regola pratica" che permette di migliorare in termini di efficienza temporale e spaziale l'esplorazione dello spazio delle soluzioni (es. A*, Greedy, RBFS, ...).

Gli algoritmi appena elencati, sono detti "classici" in quanto oltre a dare una soluzione, forniscono anche il percorso per raggiungerla (l'insieme di azioni da intraprendere). Talvolta, però, si è interessati alla sola soluzione e non al cammino che porta ad essa. In questi casi, entrano in gioco gli algoritmi di ricerca locale (es. Hill-Climbing, Simulated Annealing, ...) e online.

2.1.3 Teoria dei giochi e ricerca con avversari

Finora, tutte le definizioni fornite, riguardavano ambienti in cui vi è un solo agente. La questione si complica (e non di poco) quando gli agenti diventano due o più. In questo caso, infatti, è necessario capire se essi competono o cooperano ed agire di conseguenza. Assumendo di avere a che fare con un ambiente in cui agiscono due agenti (cosa che non porta ad alcuna perdita di generalità) che competono tra loro, studiarli utilizzando due approcci distinti:

- fissato un agente che si desidera studiare, considerare tutti gli altri come parte dell'ambiente che lo rendono non deterministico (ovvero mutabile non solo a causa di azioni dell'agente fissato);
- considerare gli agenti due entità in competizione e studiarne i possibili comportamenti facendo uso di algoritmi di ricerca per alberi di gioco.

Nel prosieguo della trattazione, si opterà per il secondo approccio, che consente di trasformare il problema in oggetto in un gioco, con le dovute semplificazioni del caso. Formalmente parlando, Russell et al. [1, Capitolo 5] sostengono che un gioco sia costituito da:

1. uno stato iniziale S_0 che indica la configurazione di partenza;
2. una funzione che indica qual è il giocatore che deve effettuare la prossima mossa;
3. l'insieme di tutte le possibili azioni legali tra cui il giocatore di turno può scegliere nello stato s ;
4. il modello di transizione che definisce lo stato risultante dall'esecuzione dell'azione a nello stato s ;
5. il test di terminazione, in grado di capire se lo stato s è terminale o meno (cioè porta alla fine della partita oppure no);
6. la funzione di utilità che definisce il punteggio da assegnare al giocatore p quando la partita termina nello stato terminale s .

Risultano essere di particolare interesse, nonché più semplici da risolvere, quei giochi in cui l'**informazione** è **perfetta**, ovvero l'ambiente è completamente osservabile, e a **somma zero** (nel momento in cui un giocatore vince, l'altro perde, e viceversa). Esempi di giochi del genere sono il tris e gli scacchi.

È stato detto che gli agenti intelligenti usano algoritmi di ricerca per trovare l'azione ottimale da eseguire. Nei giochi a informazione perfetta e a somma zero, non cambia nulla, se non per il fatto che occorre considerare anche le possibili mosse dell'avversario. Gli algoritmi di ricerca impiegati in questo tipo di contesti, prendono il nome di **algoritmi di ricerca con avversari** e vengono eseguiti sull'albero di ricerca (che prende anche il nome di albero di gioco). Come si costruisce l'albero di gioco? Risulta necessario conoscere: stato iniziale, l'insieme delle possibili azioni per ogni stato e lo stato risultante da qualsiasi mossa¹. A questo punto, sorge un'ulteriore domanda: che cosa rappresenta l'albero di gioco? Mostra tutte le azioni che il primo giocatore può effettuare e tutte le contro-mosse che il secondo giocatore può mettere in atto, se presenti. Ovviamente, se lo spazio degli stati è infinito oppure le regole del gioco consentono di ripetere una stessa posizione², l'albero di gioco sarà a sua volta infinito.

¹in questo contesto, mossa e azione sono sinonimi

²in questo contesto, posizione e stato sono sinonimi

2.1.4 Contesto matematico

Prima di analizzare i vari algoritmi di ricerca con avversari, è necessario capire perché questi funzionano, cioè perché sono in grado di fornire, con una certa accuratezza, la miglior mossa possibile in un certo istante di gioco.

La premessa indispensabile è che il gioco considerato sia a somma zero, con informazione perfetta e finito (albero di gioco finito). Detto questo, esiste un teorema, il teorema Minimax, formulato da Von Neumann negli anni Venti del secolo scorso, che sancisce l'esistenza di una strategia ottimale per ambedue gli sfidanti. Strategia ottimale non significa automaticamente garanzia di vittoria, bensì, assumendo che entrambi i giocatori giochino in maniera impeccabile (facendo cioè la scelta migliore possibile date le circostanze):

- nel caso migliore, vittoria con il valore di utilità più alto possibile;
- nel caso peggiore, pareggio (se previsto) oppure sconfitta con il valore di utilità più basso possibile.

La conclusione a cui arriva il teorema, brillantemente illustrata da Kjeldsen [2], è che il valore di utilità per chi perde o chi vince è lo stesso, ovvero: ipotizzando di avere a che fare con giocatori perfetti, l'esito del gioco rappresenta il miglior risultato possibile sia per chi vince sia per chi perde.

Ciò detto, di seguito verranno analizzati una serie algoritmi in grado di esplorare un albero di gioco, dal più banale a quello che rappresenta lo stato dell'arte oggi. Per semplificare la comprensione, il giocatore che effettua la prima mossa verrà chiamato *MAX*, mentre l'avversario sarà denominato *MIN*. *MAX* ha come obiettivo quello di massimizzare il proprio punteggio, *MIN* invece di minimizzarlo. *MAX* vince se ottiene il valore massimo possibile, *MIN* vince se ottiene il valore minimo possibile; in tutti gli altri casi, si ha un pareggio.

2.1.5 Algoritmo Greedy

Come suggeriscono Michael H. Albert et al. [3], l'algoritmo più banale che si possa utilizzare per esplorare un albero di gioco è di tipo Greedy. Questo significa che la ricerca sarà informata e guidata da un'euristica. Tale euristica può essere la

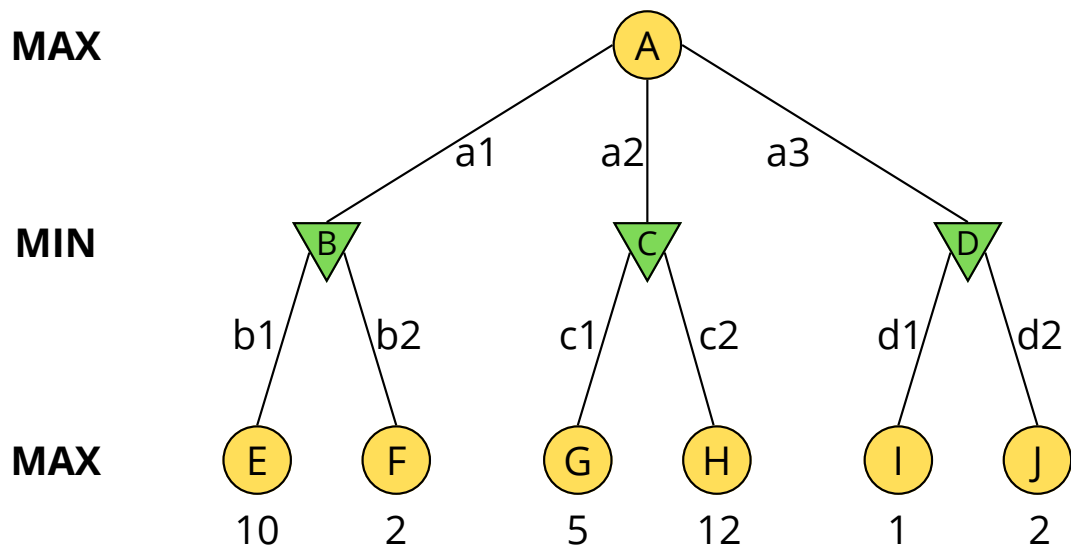


Figura 2.1: Esempio albero di gioco

stessa sia per *MIN* che per *MAX* oppure differente.

Considerando l'albero di gioco riportato in **Figura 2.1**, l'euristica che potrebbe guidare *MAX* nella ricerca della prossima mossa da effettuare è quella di scegliere l'azione che lo porta allo stato cui corrisponde la lettera dell'alfabeto più "grande" tra quelle a disposizione. *MIN*, dal canto suo, potrebbe optare per l'azione che lo porta allo stato cui corrisponde la lettera dell'alfabeto più "piccola", tra quelle a disposizione.

Quindi, se *MAX* si trova nello stato *A* e può raggiungere gli stati *B*, *C* e *D*, seguendo l'euristica opterà per la mossa *a3* che lo condurrà allo stato *D*. A questo punto, è il turno di *MIN*, che seguendo la propria euristica sceglierà la mossa *d1* che lo porterà nello stato *I*.

L'esempio ci mostra subito la limitatezza di una soluzione Greedy: ci si preoccupa solamente della prossima mossa senza guardare oltre. Con un approccio del genere, *MAX* ha perso perché *MIN* è riuscito nel suo intento di concludere la partita con il valore di utilità più basso possibile tra quelli disponibili (pari ad uno). Volendo quindi trarre delle conclusioni su questo algoritmo, esso sceglie l'azione migliore localmente, ma non è detto che questa lo sia globalmente (così è stato per *MAX*, ad esempio). Infatti, l'analisi dell'albero si basa soltanto sui nodi successivi per cui, ipotizzando che

il **branching factor**³ sia b , la complessità temporale di questo algoritmo è $O(b)$. Una ricerca del genere (detta anche "miope") porta inevitabilmente a soluzioni subottimali nella maggior parte dei casi; l'unica nota positiva risulta essere la rapidità con cui si ottiene una soluzione [1, Capitolo 3, Sezione 5].

2.1.6 Algoritmo Minimax puro

Il modo più semplice per approcciare un gioco a somma zero con informazione perfetta consiste nel risolverlo a ritroso, partendo cioè dai nodi terminali dell'albero di gioco, di cui si conoscono i valori di utilità. In base al turno, si sceglie il valore che consente di massimizzare la propria funzione di utilità (che varia da giocatore a giocatore). L'algoritmo appena descritto prende il nome di **Minimax** ed è sostanzialmente un algoritmo a forza bruta: esaminate tutte le possibili strategie di gioco⁴, sceglie quella che porta ad una soluzione ottimale [4, Capitolo 14, Sezione 3, Paragrafo 1]. Considerando l'esempio 2.1, il primo a giocare sarà *MIN*. Per ogni scelta effettuabile da *MAX*, occorre stabilire qual è la miglior scelta per *MIN*. Quindi:

- se ci si trova nello stato B , l'azione migliore da intraprendere risulta essere la $b2$, giungendo allo stato terminale F che ha un'utilità pari a 2;
- se ci si trova nello stato c , l'azione migliore da intraprendere risulta essere la $c1$, giungendo allo stato terminale G che ha un'utilità pari a 5;
- se ci si trova nello stato D , l'azione migliore da intraprendere risulta essere la $d1$, giungendo allo stato terminale I che ha un'utilità pari a 1.

Continuando ad andare a ritroso, adesso è il turno di *MAX*, che sa già che:

- con l'azione $a1$, l'avversario risponderà con l'azione $b2$;
- con l'azione $a2$, l'avversario risponderà con l'azione $c1$;
- con l'azione $a3$, l'avversario risponderà con l'azione $d1$.

³per branching factor s'intende il numero medio di figli che ha ciascun nodo dell'albero di gioco

⁴per strategia s'intende l'insieme di azioni da perseguire per arrivare alla fine del gioco

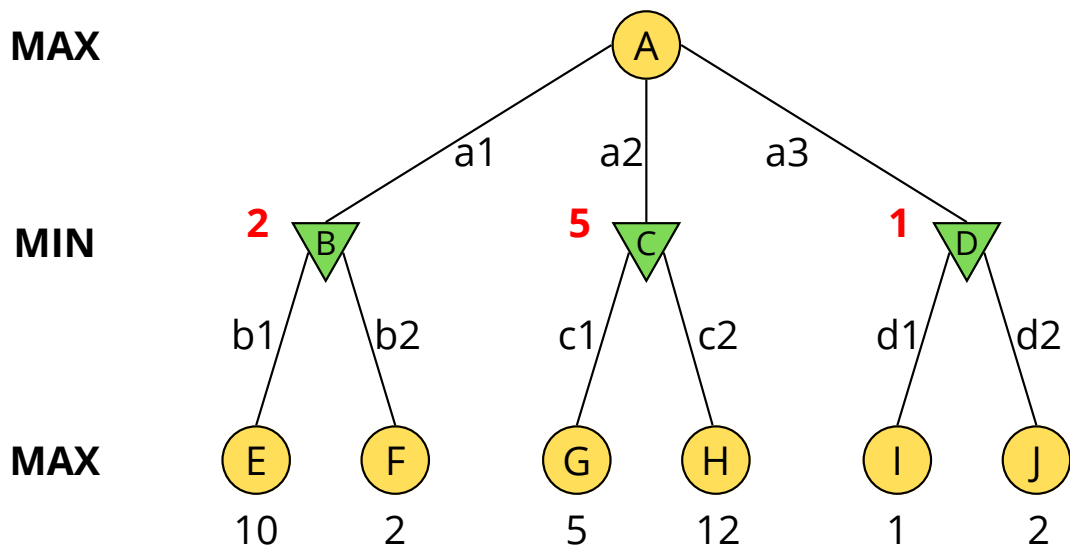


Figura 2.2: Albero di gioco dopo che *MIN* ha scelto le azioni da eseguire per ogni possibile mossa di *MAX* (in rosso)

La scelta che consente a *MAX* di massimizzare la propria funzione di utilità, assumendo che *MIN* giochi in maniera ottima, è l'azione *a2* in quanto il valore di utilità che avrà sarà pari a 5, il massimo possibile date le circostanze.

Usando questo algoritmo, è stato determinato il valore ottimale di cui si parla nel teorema Minimax di Von Neumann (vedi 2.1.4). Infatti, volendo risolvere il problema in questione usando il teorema sopra citato, basta calcolare:

$$\begin{aligned} \max\{ \min\{10, 2\}, \min\{5, 12\}, \min\{1, 2\} \} = \\ \max\{ 2, 5, 1 \} = 5 \text{ (come volevasi)} \end{aligned}$$

Quanto a pregi e difetti del Minimax "puro", se da un lato è garantita per costruzione l'individuazione della strategia ottima per ambo i giocatori, dall'altro la complessità spaziale e temporale non è delle migliori; infatti, assumendo di avere un branching factor b e una profondità massima dell'albero di gioco pari a d , la complessità temporale (e quella spaziale) è pari a $\theta(b^d)$. Questo perché anche nel caso migliore si è costretti a esplorare l'intero albero. Finché l'albero di gioco ha delle dimensioni contenute, ha senso avvalersi del Minimax; purtroppo, però, nella realtà si ha a che fare con alberi di gioco di dimensioni spropositate (si pensi agli scacchi) per cui ottenere la prossima azione da intraprendere in un tempo esponenziale rispetto alla taglia

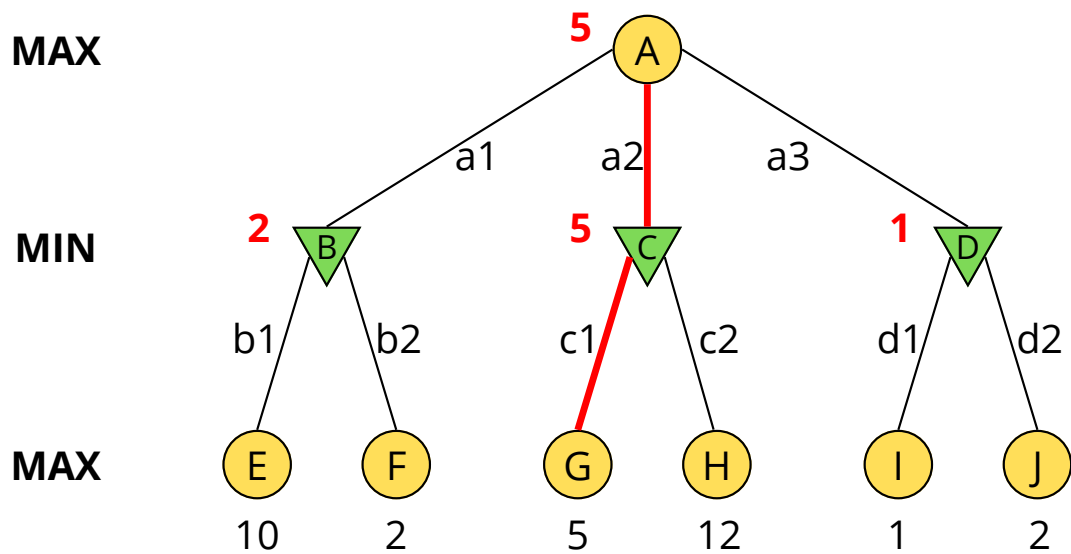


Figura 2.3: Albero di gioco dopo che MAX ha scelto l'azione da eseguire assumendo che MIN giochi in maniera perfetta

dell'input è fortemente limitante se non addirittura incalcolabile in tempi "umani" [5, Capitolo 2, Sezione 1]. Sono quindi necessarie delle migliorie.

2.1.7 Algoritmo Alpha-Beta

L'algoritmo Alpha-Beta è un primo miglioramento del Minimax puro che trae spunto dalla natura: quando un albero ha troppi rami, se ne possono potare alcuni ritenuti inutili. Stesso discorso vale per gli alberi di gioco: anziché analizzare l'intero spazio di ricerca a ritroso, si possono potare dei rami al verificarsi di particolari condizioni. Quando si analizza l'albero, si tiene traccia di due valori:

- α , che rappresenta il valore della scelta migliore (quella con il valore più alto) per MAX individuata sino a quel punto dell'analisi;
- β , che rappresenta il valore della scelta migliore (quella con il valore più basso) per MIN individuata sino a quel punto dell'analisi.

La condizione che deve verificarsi per poter potare durante l'esecuzione di tale algoritmo è la seguente: $\alpha \geq \beta$. Altra cosa molto importante da ricordare è la seguente: α può essere modificato solo durante il turno di MAX, β , invece, solo durante il turno

di MIN [6].

Si ipotizzi uno scenario del genere:

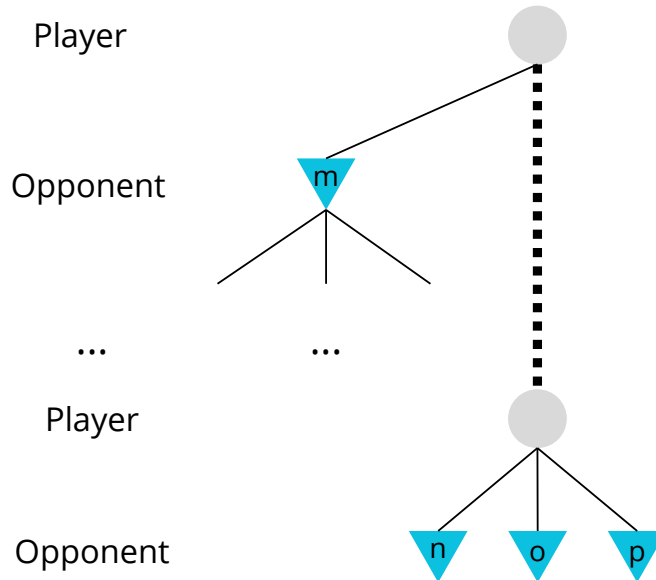


Figura 2.4: Esempio di scenario che consente la potatura in un albero di gioco

In presenza di un nodo m cui è associato un valore di utilità migliore⁵ di n (con m situato al livello del nodo padre di n o ancora più in alto), allora è possibile potare indiscriminatamente tutti gli altri rami del sottoalbero di cui n è figlio in quanto irrilevanti ai fini della ricerca del valore ottimo per il sottoalbero che ha m come figlio. Considerando l'esempio in **Figura 2.1**:

1. sebbene verrà effettuata un'esplorazione a ritroso, si associano a tutti gli stati intermedi (non foglie) i peggiori valori possibili per α e β (rispettivamente, $-\infty$ e $+\infty$);

⁵il concetto di migliore dipende da chi detiene il turno

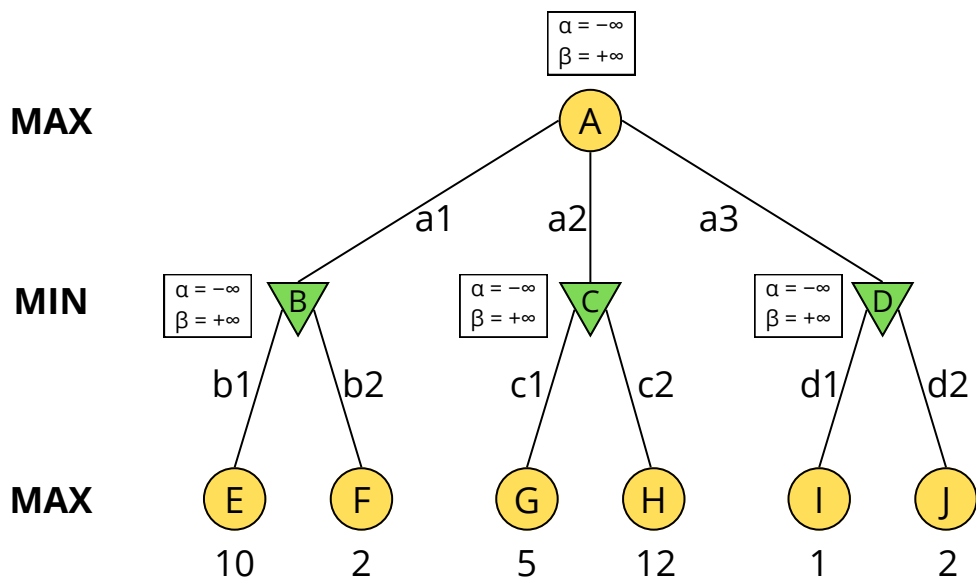


Figura 2.5: albero di partenza

2. a questo punto, si comincia dalle foglie, l'ultimo giocatore a poter compiere un'azione è *MIN*. Occorre quindi trovare il valore di β per il nodo *B* (quello più a sinistra), pari al minimo tra i valori di utilità associati alle foglie *E* ed *F*, ovvero $\min\{10, 2\} = 2$ che risulta essere più piccolo di $+\infty$, per cui $\beta_B = 2$;

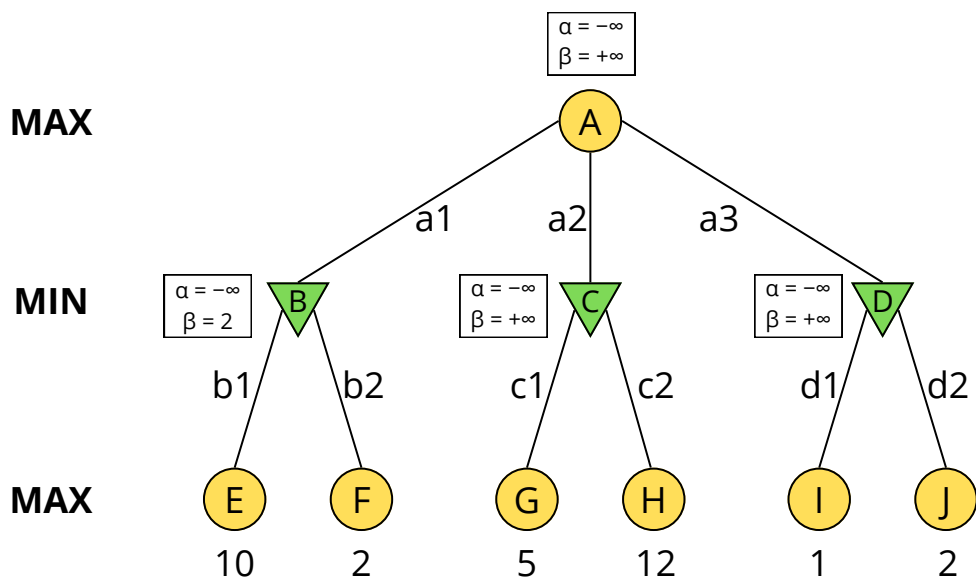


Figura 2.6: albero di gioco aggiornato dopo aver esplorato il sottoalbero con radice *B*

3. il padre di B risulta essere il nodo A : a questo livello è il turno di MAX (β_A non si tocca). È possibile aggiornare solo α_A che è pari $\max\{\alpha_A, \beta_B, \beta_C, \beta_D\} = \max\{-\infty, 2, -\infty, -\infty\} = 2$. Siccome A è la radice dell'albero di gioco, si riparte dalla foglia più a sinistra non ancora esaminata (ovvero il nodo G); in presenza di altri antenati per il nodo A , si sarebbe dovuto ripetere tale procedimento fino alla radice dell'albero di gioco (rispettando sempre i turni);

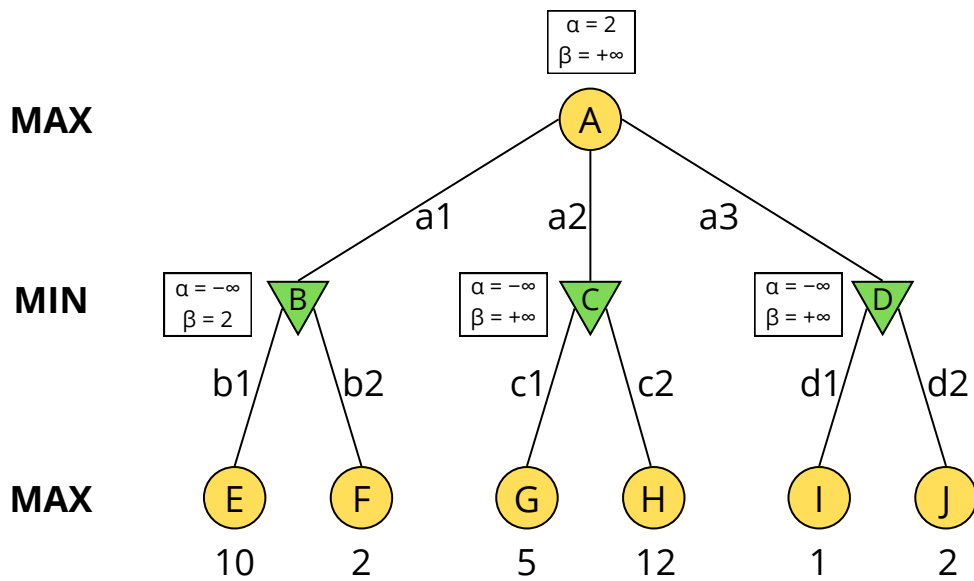


Figura 2.7: albero di gioco con path da B alla radice aggiornato

4. il nodo C eredita il valore di α da A , per cui $\alpha_C = 5$. Per individuare il valore β , va analizzata una foglia per volta, poiché potrebbe verificarsi uno scenario favorevole alla potatura. In particolare, sapendo che G ha un'utilità pari a 5, β_C , per ora, è pari proprio a 5. Sapendo che $\beta_C > \alpha_C$, non vi sono condizioni per applicare la potatura; è però possibile aggiornare α_A che si ricorda essere pari a: $\max\{\alpha_A, \beta_B, \beta_C, \beta_D\} = \max\{-\infty, 2, -\infty, -\infty\} = 2$, per cui $\alpha_A = 5$;

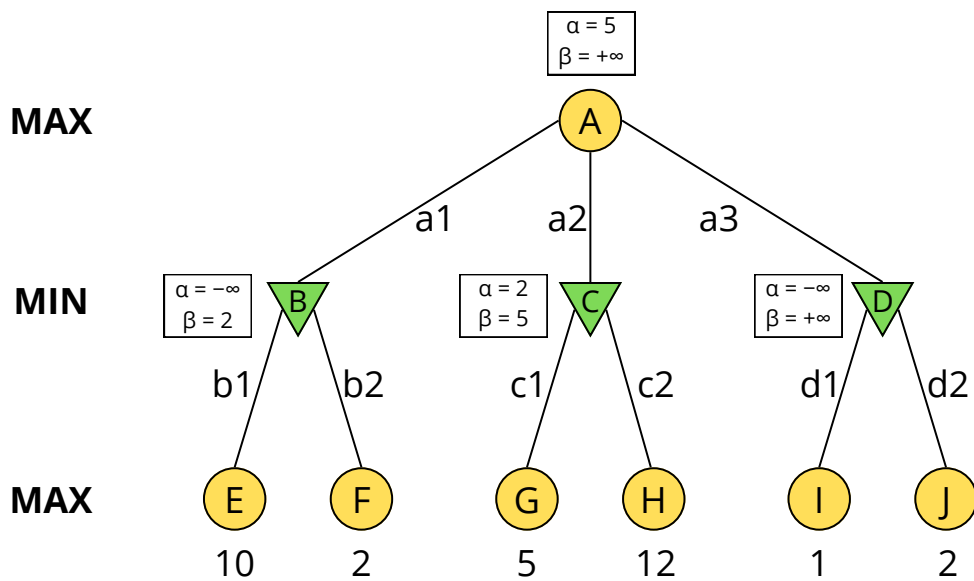


Figura 2.8: albero di gioco con path da C alla radice aggiornato

5. dopodiché, si procede con l'analisi della foglia successiva di C, ovvero H che ha un'utilità pari a 12. β_C resta invariato e quindi anche gli altri valori sul percorso verso la radice resteranno invariati;
6. tocca adesso al nodo D, che eredita il valore di α dal padre, per cui $\alpha_D = 5$. Per individuare il valore di β , va analizzata una foglia per volta, poiché potrebbe verificarsi uno scenario favorevole alla potatura. La foglia I ha un'utilità pari a 1, ragion per cui $\beta_C = 1$ (almeno per ora). A questo punto però ci si può accorgere che $\beta_D \leq \alpha_D$: possiamo evitare di analizzare tutti gli altri figli di D (i.e. potarli) in quanto vi è la certezza matematica che α_A non può migliorare.

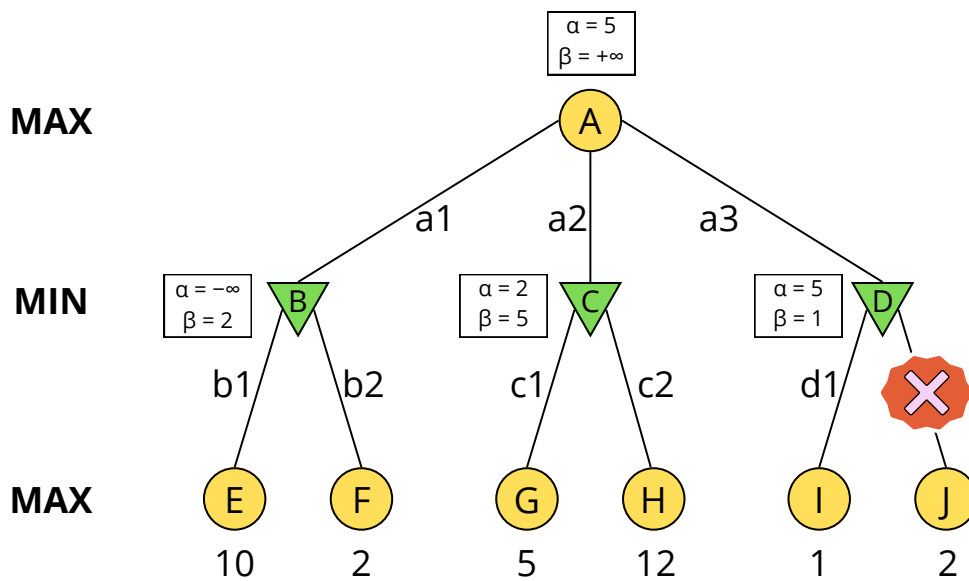


Figura 2.9: albero di gioco con path da D alla radice aggiornato e conseguente potatura

Come si può notare dall'esempio, è stata risparmiata una visita. Purtroppo non si riesce a stabilire a priori se la potatura porterà o meno una riduzione significativa dei cammini da analizzare. Nel caso migliore, infatti, si può arrivare ad una complessità temporale pari a $O(b^{d/2})^6$ quindi subesponenziale rispetto alla taglia dell'input. Nel caso peggiore, invece, si finisce per visitare l'intero albero. La complessità dipende dall'ordinamento dei nodi: siccome l'algoritmo compie essenzialmente delle DFS, se la miglior soluzione si trova sempre a sinistra per ogni path analizzato, avrò il caso migliore. Per un ordinamento casuale la complessità temporale risulta essere all'incirca $O(b^{3d/4})$ [6].

Sulla base di quanto detto, l'algoritmo Alpha-Beta può dare una mano nel ridurre la complessità temporale del problema di analizzare un albero di gioco, ma per giochi con un numero medio di mosse elevato (es. scacchi) rimane comunque inutilizzabile se non si dispone di parecchia potenza di calcolo. Con una serie di accorgimenti, è possibile ridurre ulteriormente la complessità, che resta comunque di tipo subesponenziale.

⁶dove b è il branching factor mentre d la profondità dell'albero di gioco considerato

2.1.8 Varianti Alpha-Beta

Stando a quanto riportato da Shaojia Zhang [5, Capitolo 3], ci sono due tipologie di miglioramenti che possono essere apportati all'algoritmo Alpha-Beta:

1. ottimizzare la finestra scorrevole. Questo perché inizialmente α e β valgono $-\infty$ e $+\infty$ per ogni nodo; più però si riesce a restringere tale finestra, maggiore è la possibilità di effettuare potature. A tal proposito, esistono due algoritmi:

- **ricerca con finestra di aspirazione:** si imposta una finestra iniziale $[V-e, V+e]$ ⁷. Se il valore minimax dell'albero è compreso in questo intervallo, non solo si ha la certezza di ottenere un valore ottimo ma aumenta considerevolmente anche il numero di potature possibili. Se però il β corrente è superiore a $V+e$, β viene posto a *infinity* e la ricerca riparte da capo; discorso opposto se il valore di α corrente è inferiore a $V-e$. Il problema principale è determinare la finestra iniziale e il limite di errore;
- **ricerca con finestra minima:** questo algoritmo si basa sul fatto che provare che una certa mossa è pessima richiede meno tempo che calcolare il suo punteggio. Quindi, si ipotizza che la prima mossa tra quelle disponibili sia la migliore. Se così non è, si formula una nuova ipotesi e si ripete il procedimento fino ad individuare la vera mossa migliore. Possibili implementazioni:

- (a) **PVS**⁸: si analizza il primo sottoalbero con la finestra tipica dell'algoritmo Alpha-Beta⁹. Individuata la mossa migliore, avente un punteggio V , si analizza il sottoalbero successivo con la finestra più piccola possibile, ovvero $[V, V + 1]$; se la mossa analizzata restituisce un punteggio migliore, si aggiorna la finestra e si rilancia la ricerca con i nuovi limiti, fino all'analisi completa dell'albero di gioco;

⁷dove e è un certo limite di errore

⁸Principal Variation Search, noto anche come Negascout

⁹ $[-\infty, +\infty]$

- (b) **MTD(f)**¹⁰: in questa variante, anche nel primo sottoalbero si effettua una ricerca con finestra minima $[V, V + 1]$ dove V viene scelto sulla base di una stima di quello che potrebbe essere il miglior punteggio;
- miglioramenti basati sulle ricerche precedenti¹¹. Approccio utile in alberi di gioco con dimensioni considerevoli. Principali tecniche:
 - (a) **iterative deepening** ¹²;
 - (b) **tabelle di trasposizione e confutazione**;
 - (c) **mosse killer**;
 - (d) **database con mosse di apertura e chiusura**;

Le prestazioni migliori si ottengono combinando più miglioramenti tra loro.

2.1.9 Algoritmo Minimax euristico

Nel 1950, il professor Claude E. Shannon pubblicò un articolo la cui ambizione era quella di spiegare come progettare un computer in grado di giocare a scacchi [7]. Essendo questo, infatti, un gioco a somma zero con informazione perfetta (coerente con la casistica affrontata fin qui), si accorse che l'albero di gioco da esplorare per individuare la miglior mossa possibile richiedeva tempi biblici dal punto di vista computazionale. Per questo motivo, ebbe un'intuizione, interrompere la ricerca ad una profondità fissata, né troppo piccola né troppo grande. Siccome l'albero risultante dal taglio aveva delle foglie che in realtà erano nodi intermedi, risultavano prive di utilità. Al posto quindi di usare una funzione di utilità, venne introdotta una **funzione di valutazione**, la quale forniva una stima sulla bontà della mossa. Questa variante del Minimax, prende il nome di Minimax euristico poiché la funzione di valutazione non è altro che un'euristica (nel caso degli scacchi, per esempio, si assegnava un punteggio crescente per mosse in avanti, mosse che mettevano in pericolo una pedina, mosse che mangiavano una pedina).

¹⁰Memory-enhanced Test Drive

¹¹ricerche effettuate durante l'esplorazione dell'albero di gioco nella stessa partita o turno

¹²assumendo che l'albero di gioco abbia profondità d , si comincia con l'esplorare gli stati situati ad una profondità pari ad 1; da qui, usando una funzione euristica, si stima un punteggio per ogni stato esplorato. Nell'iterazione successiva, la profondità aumenta di un'ulteriore unità ma si considera solo il cammino dalla radice al nodo con punteggio migliore dell'iterazione precedente (tutti gli altri pari livello vengono essenzialmente potati)

2.1.10 Reinforcement Learning

Il **Reinforcement Learning** (spesso abbreviato con *RL*), o anche detto "apprendimento per rinforzo", è uno dei tre paradigmi del *machine learning*¹³ in cui un agente intelligente agisce in un determinato ambiente ricevendo ricompense o penalità. Le azioni che intraprende non vengono suggerite, bensì sono frutto dell'esperienza (in presenza di uno storico delle azioni effettuate e delle conseguenze che hanno avuto) oppure della casualità¹⁴ (in assenza di una sequenza percettiva). Perciò, il concetto chiave del *RL* risulta essere tentare delle azioni e imparare dagli errori, per evitare che possano essere nuovamente commessi in futuro[8, Capitolo 1]. Oltre all'agente e all'ambiente, in un sistema di apprendimento per rinforzo occorre definire:

- la *policy* da far seguire all'agente, ovvero la strategia di gioco in questo contesto;
- il *reward signal*, ovvero il feedback numerico che l'agente riceve ad ogni azione che compie (ricompensa o penalità) che misura la bontà della stessa se considerata singolarmente (non quindi nel contesto più ampio di una strategia di gioco);
- la *value function*, che invece offre una stima sul reward nel lungo periodo partendo dallo stato selezionato dall'agente;
- il *modello* dell'ambiente (facoltativo), usato per ridurre l'apprendimento tramite esperienza diretta (i.e. sbagliando).

In un gioco a somma zero con informazione perfetta risulta ragionevole fare uso di agenti basati su *RL*, soprattutto laddove l'ambiente risulta essere molto complesso e le decisioni da prendere sono delicate [9].

2.1.11 Monte Carlo tree search

Finora sono state considerate soluzioni deterministiche al problema della ricerca con avversari. Vale però la pena menzionare anche quelle di tipo probabilistico, tra cui **MCTS** (Monte Carlo Tree Search). Un algoritmo si dice essere di tipo Monte

¹³i restanti due sono apprendimento supervisionato e apprendimento non supervisionato

¹⁴quindi, è possibile creare intelligenza attraverso la casualità

Carlo se fa uso di campionamenti o simulazioni casuali¹⁵ per stimare un valore o una soluzione (nel contesto della trattazione, l'obiettivo è stimare la bontà di una mossa rispetto alle altre). Tale stima può avvenire usando diverse tecniche, tra cui il *reinforcement learning* [1, Capitolo 5].

A questo punto, viene da chiedersi: com'è che si fanno ad estrarre informazioni utili da una simulazione? Occorre definire una **politica di rollout**, ovvero un criterio in base al quale scegliere la prossima mossa da effettuare in un dato stato. Criteri banali sono la *randomness* e l'uso di *euristiche*.

Purtroppo, la sola *rollout policy* non basta: occorre definire anche da quali stati iniziare le simulazioni e quante condurne per ciascuno di essi.

Considerando il caso specifico della **MCTS**, come facilmente intuibile dal nome, si mantiene un albero di ricerca in memoria (che non va confuso con l'albero di gioco). Ciascun nodo dell'albero **MCTS** contiene informazioni circa il numero di volte che l'algoritmo lo ha visitato e in quante simulazioni la sua visita ha portato alla vittoria (oppure il reward finale). Inizialmente è costituito dalla sola radice dell'albero di gioco (zero visite e zero vittorie) e viene di volta in volta ampliato con i seguenti passi:

1. **selezione**: partendo dallo stato iniziale dell'albero **MCTS**, sulla base della *rollout policy*, si individua un cammino che porta ad un nodo terminale (foglia dell'albero). La politica di selezione maggiormente usata è la **UCT** (upper confidence applied to trees) che classifica ogni possibile mossa usando una formula denominata **UCB1** (upper confidence bound 1) che cerca di trovare un buon compromesso tra *esplorazione* e *sfruttamento*. Per maggiori informazioni sull'argomento si rimanda alla fonte da cui sono state attinte [1, Capitolo 17]. Senza scendere troppo nei particolari, la formula necessita di conoscere quante volte è stato visitato l' i -esimo figlio del nodo di partenza e a quante vittorie ha portato la sua scelta. Per questo motivo, se lo stato iniziale ha k figli, per le prime k iterazioni si fa uso di un'altra *rollout policy* che consente la visita di tutti i figli (**UCT** entra in gioco dall'iterazione $(k+1)$ -esima).

¹⁵dette anche *rollout*

2. **espansione**: al nodo terminale individuato nella fase precedente si aggiunge il figlio scelto nell'albero di gioco (che avrà inizialmente zero visite e zero simulazioni vincenti);
3. **simulazione**: sull'albero di gioco vero e proprio si esegue una simulazione partendo dal nodo appena aggiunto nell'albero **MCTS** seguendo la *policy* stabilita precedentemente e si salva l'esito della stessa (vittoria o sconfitta per *MAX*);
4. **retropropagazione**: sulla base dell'esito della simulazione si aggiornano i valori associati ai nodi dell'albero **MCTS**. In particolare:
 - se *MAX* ha vinto, i nodi *MAX* dovranno incrementare di un'unità sia le visite che le simulazioni vincenti mentre i nodi *MIN* dovranno incrementare di un'unità solamente il numero di visite;
 - se *MAX* ha perso, i nodi *MIN* dovranno incrementare di un'unità sia le visite che le simulazioni vincenti mentre i nodi *MAX* dovranno incrementare di un'unità solamente il numero di visite.

Ciò che si può notare è che il tempo necessario per calcolare una simulazione è lineare nella profondità dell'albero di gioco, poiché ad ogni punto di scelta, si effettua una sola mossa. Terminate le iterazioni, viene il criterio di scelta della mossa migliore non è univoco: si può scegliere il nodo maggiormente visitato, quello che ha un rapporto tra vittorie e visite maggiore oppure quello che ha permesso di ottenere il maggior numero di vittorie [1, Capitolo 5, Sezione 4]. Stando a quanto riportano numerosi esperimenti, *MCTS* risulta avere performance migliori rispetto all'algoritmo Alpha-Beta (anche con tutti gli accorgimenti del caso per ridurre lo spazio di ricerca) [5].

2.1.12 Algoritmo AlphaZero

AlphaZero è un algoritmo sviluppato da DeepMind e rilasciato nel 2017. Rappresenta lo stato dell'arte per quanto riguarda l'AI applicata ai giochi (e quindi anche quelli a somma zero con informazione perfetta, che ne sono una versione semplificata). Trattasi di un algoritmo di tipo **tabula rasa** (ovvero parte completamente da zero,

senza conoscenze pregresse o dati di addestramento provenienti da esperti umani, e apprende esclusivamente interagendo con l'ambiente).

Anziché fare uso di funzioni di valutazione definite a mano e di euristiche per ordinare le mosse (tecniche di cui ci si avvaleva fino a quel momento, anche nelle macchine più sofisticate), *AlphaZero* sfrutta una **DNN** (Deep Neural Network) che prende in input lo stato corrente s e in output restituisce un vettore di probabilità p la cui i -esima componente indica quanto è promettente la i -esima mossa secondo la mossa e uno scalare v che stima la probabilità di vittoria del giocatore nello stato s . Invece di effettuare una ricerca con l'algoritmo *Alpha-Beta* facendo uso di miglioramenti domain-specific, *AlphaZero* sfrutta l'algoritmo *MCTS* (che è general-purpose). Nella fase di **selezione** della ricerca ad albero Monte Carlo viene scelto quel nodo che ha meno visite, più vittorie (o score medio più alto) e probabilità p_i maggiore (fornita dalla rete neurale). In questo modo, si favorisce l'esplorazione anziché lo sfruttamento. Alla fine, l'algoritmo restituisce un vettore π con la distribuzione delle probabilità per le varie mosse disponibile in un certo momento della partita. Tale distribuzione può essere:

- **proporzionale**;
- **greedy**: solo una mossa ha valore 1, tutte le altre 0 (si forza l'azione da compiere);

Altra caratteristica fondamentale è che, in questo caso, *MCTS* non fa *rollout* casuali, ma si basa solo ed esclusivamente sui dati che la rete neurale mette a disposizione. I parametri che usa la rete neurale inizialmente hanno valori casuali; man mano che l'addestramento va avanti (tramite *self-play*) si apportano i dovuti aggiustamenti per minimizzare una certa funzione di perdita (che, per semplificare le cose, può essere vista come la probabilità di sconfitta) [10].

2.2 GEOGRAPHY ILLITERACY E LEARNING BY GAMIFICATION COME POSSIBILE STRUMENTO DI CONTRASTO

2.2.1 Geography Illiteracy

Di solito, alla prima lezione di geografia nella vita di un individuo, si analizza il significato di questo termine e si scopre che deriva dal greco: vuol dire "descrizione della Terra". Di conseguenza, quando si parla di *geography illiteracy* (in italiano, analfabetizzazione geografica) si intende l'incapacità di saper "descrivere la Terra" e quindi l'avere gravi lacune in termini di zonazione, localizzazione e comprensione delle relazioni spaziali tra i diversi elementi che compongono il pianeta.

L'analfabetizzazione geografica non riguarda solo il non sapere "dove si trova cosa" sulla mappa, ma implica anche la difficoltà nel comprendere fenomeni fisici (come il clima o l'orografia), processi socio-economici (come le migrazioni o la distribuzione delle risorse) e interconnessioni globali [11].

Uno studio del 2002, condotto da **National Geographic Education Foundation** su un campione di 3000 studenti, di età compresa tra 18 e 24 anni, provenienti da nove paesi distinti (Canada, Francia, Germania, Giappone, Italia, Messico, Regno Unito, Svezia e USA), ha evidenziato quanto siano scarse le capacità dei giovanissimi in ambito geografico. Tra i vari task proposti, ve n'era uno in cui bisognava localizzare un certo numero di stati (più o meno noti, in quanto citati spesso e volentieri al telegiornale per vari motivi, tra cui guerre, proteste, ...) sulla mappa. Ciò che emerge dai dati è che asiatici e nordamericani hanno serie difficoltà di zonazione rispetto alla controparte europea, che comunque raggiungono a stento la sufficienza. Volendo però porre l'enfasi sui risultati riportati dai partecipanti italiani al sondaggio, risulta che:

- meno del 50% è in grado di individuare i paesi dell'Est Europa, con un misero 17% nel caso della Bulgaria;
- più del 70% sa riconoscere i paesi che hanno un ruolo di primo piano nelle dinamiche internazionali;

- il 97% ha individuato correttamente l'Italia.

Per maggiori dettagli sul sondaggio, si rimanda al report completo [12].

Considerare un report del 2002 nel 2025 per constatare che la *geography illiteracy* è una piaga sociale potrebbe sembrare anacronistico; però, se si considera che nella maggior parte degli individui le conoscenze geografiche di cui sono in possesso sono state acquisite tra i banchi, potrebbe essere ragionevole pensare che la situazione, in 23 anni, possa essere solo peggiorata. Questo perché:

1. il susseguirsi delle riforme dell'istruzione ha portato alla riduzione del monte orario dedicato alla geografia. In particolare, nei licei la si accorpa alla storia in un'unica materia mentre negli istituti tecnici e professionali le si dedica un'ora a settimana il primo anno e poi scompare dal quadro orario;
2. l'aumento dell'uso di strumenti di navigazione digitale (come Google Maps) ha ridotto la necessità di memorizzare e comprendere la geografia fisica e politica, favorendo un approccio "usa e getta" delle informazioni spaziali;

D'altro canto, è pur vero che oggi è più semplice viaggiare rispetto a 20 anni fa, quindi, volenti o nolenti, anche in maniera inconscia, si finisce per apprendere qualcosa sul posto in cui si va, dove si trova, i piatti tipici, i monumenti, le bellezze naturali e così via. Inoltre, la diffusione di contenuti multimediali e documentari a tema geografico attraverso piattaforme di streaming e social media permette, anche senza un percorso scolastico strutturato, di entrare in contatto con informazioni e curiosità sui diversi Paesi e culture, contribuendo ad ampliare – seppur in modo disomogeneo – il bagaglio geografico delle persone.

2.2.2 Learning by Gamification

La *gamification* è l'applicazione di elementi, dinamiche e meccanismi tipici dei giochi (come punti, livelli, sfide, premi, classifiche) in contesti non ludici, con l'obiettivo di aumentare il coinvolgimento, la motivazione e la partecipazione degli utenti. Un ambito in cui sta prendendo sempre più piede questo tipo di approccio è quello scolastico. Infatti, durante la pandemia, per aumentare il grado di coinvolgimento degli studenti, costretti a seguire le lezioni da casa, molti docenti hanno introdotto quiz,

giochi a premi per cercare di rendere quanto più interattive possibili le loro spiegazioni. Terminata l'emergenza, si è comunque deciso in alcuni casi di mantenere questa metodologia di insegnamento perché molti studi ne hanno certificato la bontà. Nel caso specifico della geografia, essendo questa una materia scarsamente considerata anche dal punto di vista accademico, insegnarla tramite *gamification* potrebbe far appassionare più studenti e migliorarne il rendimento, sebbene la sua adozione resti ostacolata non solo da vincoli materiali, ma anche da una forte inerzia istituzionale [13].

2.2.3 Stato dell'arte

Negli ultimi anni sono emerse numerose piattaforme digitali che sfruttano tecniche di *gamification* per migliorare la conoscenza geografica, combinando elementi di carattere ludico con contenuti didattici. Le più in voga oggi risultano essere:

- **GeoGuessr**: lanciata nel 2013, è una piattaforma che utilizza immagini di Google Street View per "posizionare" virtualmente l'utente in un punto casuale del globo. Il giocatore deve indovinare la località esatta analizzando dettagli visivi come paesaggi, segnaletica stradale o architettura. Il sistema assegna punti in base alla precisione della risposta e al tempo impiegato, favorendo lo sviluppo di competenze di osservazione, interpretazione e orientamento spaziale;
- **Seterra**: disponibile sia in versione web sia come applicazione mobile, offre oltre 400 attività interattive su mappe politiche, fisiche e tematiche. Gli utenti possono esercitarsi nel riconoscimento di paesi, capitali, bandiere e caratteristiche fisiche. La progressione a livelli, i punteggi e i timer stimolano la competizione e il miglioramento personale;
- **World Geography Games**: pensata per un pubblico scolastico, questa piattaforma propone giochi a tema geografico suddivisi per continente, tipo di informazione (stati, capitali, fiumi, montagne) e difficoltà;
- **Kahoot!**: pur non essendo dedicata esclusivamente alla geografia è ampiamente utilizzata dai docenti per creare quiz interattivi su misura. La modalità com-

petitiva in tempo reale, la possibilità di inserire immagini e mappe e l'uso di classifiche istantanee favoriscono un elevato coinvolgimento da parte degli studenti¹⁶.

Ovviamente, questi sono solo alcuni degli esempi sul tema, il panorama delle risorse digitali "gamificate" per l'apprendimento della geografia è in continua espansione.

¹⁶basti pensare che è stato usato anche in alcuni corsi della laurea in informatica all'Università degli Studi di Salerno

Proposta di gioco: BorderGuessr

In questo capitolo verrà presentato il gioco sul quale verrà condotta l'analisi empirica con i vari algoritmi citati nel capitolo precedente: BorderGuessr. Si partirà dal descrivere come è nato, in cosa consiste e verrà mostrato anche un esempio pratico. Il tutto ad un livello di astrazione piuttosto elevato, senza quindi scendere nei dettagli implementativi. Infine, verranno brevemente illustrate le motivazioni che lo rendono un buon caso di studio.

3.1 Genesi

Sin da quando ero bambino, ho sempre avuto un particolare interesse per la geografia, cosa che indubbiamente mi ha trasmesso mio nonno materno. Ciò che più mi incuriosiva erano i confini tra gli stati, passati e odierni, con un occhio di riguardo per le triplici frontiere. Crescendo, questa passione è rimasta e per "mantenermi allenato" spesso volte passavo il mio tempo libero su siti come GeoGuessr, Seterra... Durante gli ultimi anni delle superiori, per ammazzare il tempo durante le sostituzioni o i cambi d'ora, con un amico ero solito giocare a una sorta di "geografia a catena": uno di noi diceva il nome di uno Stato e l'altro doveva rispondere con un altro Stato

che iniziasse con l'ultima lettera del precedente¹. Ad esempio, se io dicevo Italia, il mio amico poteva dire Afghanistan, e così via, finché qualcuno non rimaneva senza idee. Dagli Stati, siamo poi passati alle città italiane, poi alle capitali e infine alle città non capitali... Questo semplice passatempo, oltre a essere un esercizio di memoria e rapidità, alimentava la mia curiosità verso la geografia politica. Il problema di tutte queste varianti è che il nesso tra una mossa e l'altra era di carattere puramente alfabetico e non propriamente "geografico". Così un giorno, mentre mi scervellavo sulla prossima modifica da apportare, ebbi un'intuizione: perché non sfruttare i confini? Data quindi una città, una regione o uno Stato, la mossa successiva sarebbe potuta essere una città, una regione o uno Stato confinante con quello appena detto. Ancora non lo sapevo, ma avevo gettato le basi per il mio progetto di Fondamenti di Intelligenza Artificiale (che si sarebbe poi trasformato anche in progetto di tesi). I semestre anno accademico 2024/2025: tra i vari corsi a scelta c'è quello di *FIA*², lo inserisco tra i miei esami e credo che mai scelta sia stata più azzeccata. Per superarlo, occorre sviluppare un progetto inerente le tematiche affrontate durante il corso, che può essere diviso in due macro-aree: algoritmi di ricerca e machine learning. Le idee sono tante ma nessuna sembra soddisfare a pieno le mie aspettative. L'obiettivo era sviluppare qualcosa che coniugasse gli algoritmi di ricerca con la mia passione per la geografia. Pensando e ripensando, mi ricordo di questo giochino (che ancora non aveva un nome) e sembra avere tutte le caratteristiche di un gioco a somma zero con conoscenza perfetta. Insieme agli altri colleghi del gruppo creato per realizzare il progetto, decidiamo di battezzarlo **BorderGuesser**.

3.2 Descrizione

Il funzionamento di **BorderGuesser** è molto semplice ed intuitivo: data una Nazione (presumibilmente scelta a caso), il primo giocatore è chiamato a fornire uno Stato confinante con essa. L'avversario dovrà indicare, a sua volta, uno Stato confinante non ancora menzionato e così via. Il gioco termina quando uno dei due sfidanti sbaglia oppure quando si arriva ad una situazione di blocco. Si verifica un "blocco"

¹sono state sperimentate anche versioni in cui l'iniziale doveva coincidere con la penultima o la terzultima lettera

²Fondamenti di Intelligenza Artificiale

quando il giocatore che detiene il turno non può menzionare alcuno Stato per il semplice fatto che sono stati tutti già utilizzati nel corso della partita.

Per sciogliere qualsiasi dubbio sul funzionamento, di seguito vengono proposte due partite di esempio:

- **partita 1:** la partita inizia dalla Francia³; il primo giocatore sceglie l'Italia⁴, poi il secondo risponde con l'Austria⁵. A questo punto, il primo tenta con la Spagna, ma poiché l'Austria non confina con essa, il gioco si interrompe e il secondo giocatore conquista la vittoria;
- **partita 2:** la partita inizia da San Marino⁶; il primo giocatore sceglie l'Italia⁷, mossa corretta; il secondo risponde con Città del Vaticano, ma l'unico Stato confinante con essa è proprio l'Italia, già nominata. Il primo giocatore non ha quindi mosse disponibili (situazione di blocco) e il secondo si aggiudica la vittoria.

3.3 Rilevanza per lo studio

BorderGuessr rappresenta un caso di studio particolarmente interessante per l'analisi e l'applicazione di algoritmi di ricerca e per l'esplorazione di concetti fondamentali della teoria dei giochi. La struttura del gioco, basata sui confini tra Stati, può essere facilmente modellata come un grafo, dove ogni nodo corrisponde a uno Stato e ogni arco a un confine condiviso. Questa rappresentazione rende lo spazio di gioco definito e finito, consentendo di esplorare in modo sistematico tutte le mosse possibili e di applicare algoritmi di ricerca classici come Minimax o Alpha-Beta.

Essendo un gioco a somma zero con conoscenza perfetta, ogni vantaggio acquisito da un giocatore corrisponde a una perdita per l'altro. Questo lo rende ideale per studiare strategie ottimali e valutare diverse politiche di scelta delle mosse in un contesto

³possibili mosse: Brasile, Belgio, Lussemburgo, Germania, Svizzera, Italia, Andorra, Spagna, Monaco

⁴possibili mosse: San Marino, Città del Vaticano, Slovenia, Austria, Svizzera. La Francia non può più essere utilizzata

⁵possibili mosse: Svizzera, Liechtenstein, Germania, Cechia, Ungheria, Slovacchia, Slovenia. L'Italia non può più essere utilizzata

⁶possibili mosse: Italia

⁷possibili mosse: Città del Vaticano, Francia, Svizzera, Austria, Slovenia. San Marino non può più essere utilizzata

completamente deterministico. Allo stesso tempo, il gioco ha un valore educativo e cognitivo, stimolando la memoria geografica e la rapidità di pensiero, ponendosi l'ambizioso obiettivo di contrastare la *geography illiteracy*.

Grazie a queste caratteristiche, **BorderGuessr** costituisce un perfetto ponte tra teoria e implementazione: le regole semplici e la modellazione chiara permettono di sviluppare un progetto completo, dalla rappresentazione dei dati geografici all'implementazione dei meccanismi di gioco, fino all'applicazione e al confronto degli algoritmi di ricerca, creando così un caso di studio efficace per il campo dell'intelligenza artificiale.

Progettazione e implementazione

Obiettivo di questo capitolo è illustrare le principali scelte di progettazione e implementazione effettuate per sviluppare BorderGuessr, opportunamente motivate. Si seguiranno due filoni: quello preparatorio per la successiva analisi empirica e quello atto a favorire l'apprendimento della geografia, più precisamente dei confini, anche per i meno esperti.

4.1 PROGETTAZIONE

4.1.1 Specifica PEAS

Prima di andare ad illustrare le scelte progettuali ed implementative, essendo l'applicativo che si vuole realizzare di tipo *agent-based*, risulta utile definire formalmente le sue caratteristiche attraverso il modello **PEAS** (Performance measure, Environment, Actuators, Sensors). Questa rappresentazione consente di delineare in maniera chiara e sintetica gli obiettivi del sistema, il contesto in cui opera, le modalità di interazione e i criteri con cui verrà valutato. In particolare:

- **performance measure:** concludere la partita nel minor numero di mosse valide possibili;

- **environment**: insieme di tutti gli Stati del mondo;
- **actuators**: basta un unico attuatore che consenta di fornire al motore di gioco uno Stato;
- **sensors**: sono necessari i seguenti sensori:
 1. per rilevare gli Stati già menzionati;
 2. per rilevare gli Stati ancora disponibili;
 3. per segnalare lo stato corrente;
 4. per tenere traccia a quale giocatore appartiene il turno corrente;

4.1.2 Formalizzazione del problema

Essendo **BorderGuesser** un gioco, come descritto nella **Sezione 2.1.3**, possiede le seguenti componenti:

- **stato iniziale**: dal punto di vista teorico si tratta di una nazione scelta casualmente; dal punto di vista implementativo invece, siccome a sfidarsi sono un agente software ed un agente umano, lo stato iniziale sarà una lista vuota, facendo così corrispondere ad una mossa dell'agente software la generazione casuale dello Stato di partenza;
- **funzione TURNO(s)**: indica qual è il giocatore che deve compiere un'azione nello stato s ;
- **insieme AZIONI(s)**: insieme degli Stati confinanti con s ancora non menzionati;
- **funzione RISULTATO(s, a)**: funzione che restituisce lo stato in cui si arriva se da s si compie l'azione a (modello di transizione). Lo stato può essere il nome di una nazione accettabile, non accettabile oppure un stringa cui non è possibile associare alcuna nazione (errore di battitura, ortografico...);
- **funzione TERMINALE(s)**: funzione booleana che restituisce vero se la partita termina nello stato s , falso altrimenti. Nel caso specifico di **BorderGuesser**, una partita termina se viene inserita un stringa non associabile ad alcuno Stato

oppure se la nazione digitata è stata già menzionata precedentemente. In questi casi si ha una vittoria per l'avversario. Se invece viene fornito in input uno Stato che confina con quello corrente e tutte le nazioni che confinano con esso sono state già utilizzate nel corso della partita, si ha una vittoria per il giocatore corrente;

- **funzione UTILITÀ(s, p):** se la partita termina nello stato s , questa funzione restituisce il payoff associato ad esso per il giocatore p . Una prima versione naïve di questa funzione potrebbe assegnare utilità 1 se s porta ad una vittoria del giocatore p , 0 altrimenti (nel seguito verrà mostrata una variante alternativa).

Sulla base di quanto appena riportato, si ha a disposizione tutto l'occorrente per implementare **BorderGuessr** facendo uso dello pseudocodice.

4.1.3 Contesto e strumenti

Gli scopi di questo lavoro di tesi sono essenzialmente due:

1. da un lato, effettuare una sorta di benchmark dei vari algoritmi utilizzati nei giochi a somma zero con conoscenza perfetta nel caso specifico di **BorderGuessr** e capire quale di essi risulta essere il più adatto in questo contesto;
2. dall'altro, far sì che il gioco proposto aiuti a contrastare l'analfabetismo geografico e porti dunque le persone ad appassionarsi a questa materia, sempre attuale e per niente banale.

Sebbene le modalità di sviluppo varino in base all'obiettivo, esistono alcuni elementi comuni:

- **l'ambiente di sviluppo (IDE):** per l'applicativo di benchmarking si utilizzerà IntelliJ IDEA nella sua versione originale, mentre per l'applicativo ludico verrà impiegato Android Studio, la versione adattata da Google dell'IDE di JetBrains.
- **il linguaggio di programmazione:** in entrambi i casi si userà Kotlin, linguaggio moderno e completamente interoperabile con Java, oggi il più diffuso nello sviluppo di applicazioni Android.

4.1.4 Criteri per la scelta degli Stati e dei confini

Siccome lo scopo di questo lavoro di tesi è di carattere puramente **tecnico** (e perciò **apolitico**), è stato necessario mantenere una certa imparzialità anche nella scelta di quali entità considerare come Stati sovrani a tutti gli effetti (con il conseguente diritto di essere incluse in `BorderGuessr`) e quali invece no.

Il criterio adottato è stato molto semplice per la stragrande maggioranza dei casi: sono Stati *de iure* e *de facto* tutti i membri dell'ONU (che ad oggi, settembre 2025, conta 193 membri)¹.

Esistono però anche Stati che non fanno parte dell'ONU. In questi casi, per garantire la massima imparzialità è stato adottato il seguente criterio: se più della metà dei membri dell'ONU riconosce uno Stato non membro, allora è da considerarsi *de facto* uno Stato sovrano. Ciò quindi ha permesso di includere nel gioco:

- **Città del Vaticano**, che intrattiene relazioni diplomatiche con 181 membri dell'ONU;
- **Stato di Palestina**, che al marzo 2025 era formalmente riconosciuto da 147 membri dell'ONU (numero destinato a salire nei prossimi mesi);
- **Kosovo**, riconosciuto da 108 membri.

Esclusione eccellente risulta quindi essere **Taiwan** che venne espulso dall'ONU nel 1971 e attualmente intrattiene relazioni diplomatiche soltanto con 11 Stati membri ².

4.2 IMPLEMENTAZIONE PER L'ANALISI EMPIRICA

4.2.1 Descrizione astratta e pseudo-codice

Una partita di `BorderGuessr` coinvolge:

1. una lista di paesi, che definisce il dominio del gioco;
2. due algoritmi decisionali, che si alternano nell'effettuare le mosse;

¹la lista completa è consultabile al seguente link: https://it.wikipedia.org/wiki/Stati_membri_delle_Nazioni_Unite

²molti di più se si considerano canali non ufficiali e non la sola presenza di ambasciate sul proprio territorio

3. una board di gioco, che tiene traccia dei turni e delle mosse già effettuate.

Il gioco inizia scegliendo casualmente uno Stato (tra quelli disponibili) e tale mossa viene attribuita al primo algoritmo. Dopodiché, a turni alterni, ciascun algoritmo, partendo dal secondo, seleziona la prossima mossa, basandosi sulla situazione corrente. Il processo continua fino a quando non sono più disponibili mosse valide, ovvero si è giunti a conclusione della partita.

Volendo essere più formali, è possibile riassumere quanto detto nel seguente pseudo-codice:

```
ALGORITMO BorderGuessr:  
  1. scegli uno Stato a caso e aggiungilo alla sequenza di  
    gioco  
  2. imposta il turno per il secondo algoritmo  
  3. finché ci sono mosse valide disponibili:  
    3a. l'algoritmo di turno calcola la miglior mossa  
        dal suo punto di vista  
    3b. aggiungi la mossa alla sequenza  
    3c. passa il turno all'altro algoritmo  
  4. restituisci la sequenza delle mosse effettuate
```

4.2.2 Descrizione tecnica

Una partita non è altro che un oggetto della classe *BorderGuessrGame*, che conserva informazioni quali gli algoritmi che si stanno sfidando, la board di gioco ed il file JSON da cui leggere l'elenco degli Stati ³.

Un algoritmo è un oggetto di una classe che implementa l'interfaccia *Algorithm* e fornisce una implementazione specifica del metodo *findBestMove()*, che prende in input la board corrente e restituisce un oggetto di tipo *Move* (contenente la prossima mossa da eseguire).

La board è un oggetto della classe *BorderGuessrBoard* e memorizza informazioni quali il turno, la lista degli Stati (salvati in oggetti di tipo *Country*) e la lista delle mosse già

³è stata preferita una lettura da file rispetto che da database per questioni di efficienza

effettuate (salvate in oggetti di tipo *Node*).

Com'è che quindi si svolge una partita, in un'implementazione del genere, di **BorderGuessr**?

Innanzitutto, occorre creare un oggetto *BorderGuessrGame*, che provvede a sua volta ad inizializzare una *BorderGuessrBoard*, ovvero a:

- passarle la lista dei paesi coinvolti nella partita;
- inserire nella lista delle mosse un paese scelto a caso, che vale come mossa del primo giocatore;
- impostare il turno in favore del secondo algoritmo coinvolto nella partita.

A questo punto, invocando il metodo *play()*, ha inizio la partita vera e propria: finché non si raggiunge uno stato di terminazione (mancanza di mosse da poter effettuare), a turni alterni, si esegue la mossa restituita in output dall'implementazione specifica della *findBestMove()* dell'algoritmo che detiene il turno. Eseguire una mossa significa aggiungere alla lista di *Node* della board un nuovo elemento che memorizza lo Stato contenuto nella *Move* e i suoi vicini immediatamente raggiungibili, non ancora menzionati nel gioco (cioè non ancora salvati nella lista di nodi).

4.2.3 Esempio

Si supponga che la lista degli Stati appartenenti al dominio sia la seguente:

Nome	Stati confinanti
Bielorussia	Lettonia, Lituania, Polonia
Estonia	Lettonia
Lituania	Bielorussia, Lettonia, Polonia
Lettonia	Bielorussia, Estonia, Lituania
Polonia	Bielorussia, Lituania

Tabella 4.1: Dominio del gioco nell'esempio greedy

Immaginando che si stiano sfidando l'algoritmo X e l'algoritmo Y, sia la **Lituania** il paese scelto a caso e aggiunto alla lista delle mosse già effettuate. L'albero di gioco risulta essere il seguente:

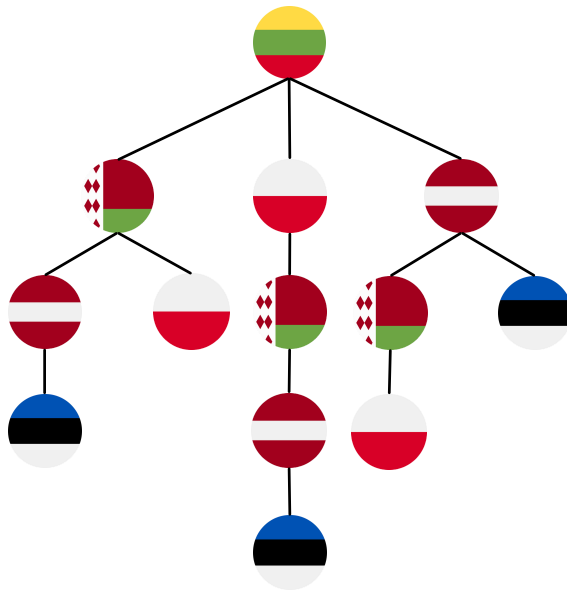


Figura 4.1: Albero di gioco con Stato iniziale Lituania

A questo punto, vista la presenza di mosse valide, tocca a Y che, fatte le dovute valutazioni, identifica in **Lettonia** la mossa migliore possibile. Di conseguenza, adesso la lista delle mosse effettuate sarà costituita da:

- Lituania
- Lettonia

L'albero di gioco cui dovrà fare riferimento X sarà invece:

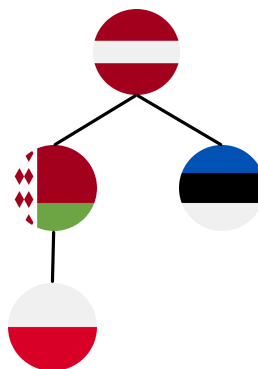


Figura 4.2: Albero di gioco a partire da Lettonia

Essendovi ancora delle mosse valide disponibili, tocca a X che, fatte le dovute valutazioni, identifica in **Estonia** la mossa migliore possibile. Di conseguenza, adesso la lista delle mosse effettuate sarà costituita da:

- Lituania
- Lettonia
- Estonia

L'albero di gioco cui dovrà fare riferimento Y sarà invece:



Figura 4.3: Albero di gioco a partire da Estonia

Siccome non sono disponibili mosse valide per Y , la partita si conclude con una vittoria per X e in output viene restituita la lista delle mosse che hanno portato a tale esito.

4.2.4 Algoritmo Greedy

Come già discusso nella relativa sezione nel capitolo relativo allo stato dell'arte, un qualsivoglia algoritmo Greedy viene guidato nella ricerca da una funzione euristica. Nel caso specifico di **BorderGuessr**, si è scelto di utilizzare come criterio di riferimento il numero di Stati confinanti. Di conseguenza, l'algoritmo sceglierà sempre, tra le mosse disponibili, quella che porta allo Stato con il numero più basso di vicini consentiti al turno successivo.

A tal proposito, supponendo che il dominio sia quello della **Tabella 4.1** e il paese scelto a caso sia la Lituania. Considerando l'albero di gioco di **Figura 4.1**, le possibili mosse sono:

- **Bielorussia**, dove $h(\text{Bielorussia}) = 2$ (la Bielorussia confina con Lettonia, Lituania e Polonia ma sono ammissibili solo Lettonia e Polonia);

- **Polonia**, dove $h(Polonia) = 1$ (la Polonia confina con Bielorussia e Lituania ma è ammissibile la sola Bielorussia);
- **Lettonia**, dove $h(Lettonia) = 2$ (la Lettonia confina con Estonia, Bielorussia e Lituania ma sono ammissibili solo Bielorussia ed Estonia).

Quindi, l'algoritmo Greedy sceglierà la Polonia. In questo caso, però, ciò porterà ad una sconfitta certa, come si può notare anche in figura:

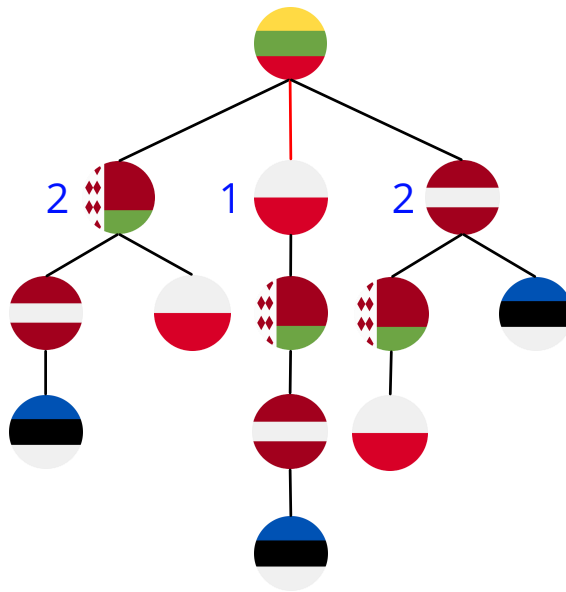


Figura 4.4: Esempio scelta effettuata da Greedy

4.2.5 Algoritmo Minimax standard e Alpha-Beta pruning

L'algoritmo Minimax standard e la sua versione ottimizzata Alpha-Beta sono da considerarsi come dei giocatori perfetti in quanto in grado di effettuare sempre la miglior mossa possibile. Come si fa però a stabilire se una mossa è migliore di un'altra? Occorre esplorare l'intero albero di ricerca e assegnare agli stati terminali un valore di utilità tramite un'apposita funzione $f(x)$.

Nel caso specifico di **BorderGuessr**, essa risulta essere la seguente:

- se la foglia in questione si trova ad una profondità⁴ dispari, $f(x) = \text{numero totale di Stati} - \text{profondità}$;

⁴la convenzione utilizzata è che il nodo radice si trova a profondità 1, anche se talvolta alcuni libri tendono ad assegnarle profondità 0

- altrimenti, $f(x) = -(\text{numero totale di Stati} - \text{profondità})$.

Sarebbe stato sicuramente più semplice assegnare valore 1 ai nodi terminali che portano ad una vittoria di MAX e -1 a quelli che portano ad una sua sconfitta (pareggiare non è possibile). L'obiettivo però non è solo vincere, ma farlo nel numero minore di mosse possibili, ragion per cui è stata preferita la funzione di utilità descritta poc'anzi. Per una migliore comprensione, si consideri il seguente esempio:

Nome	Stati confinanti
Germania	Polonia
Estonia	Lettonia
Lituania	Lettonia, Polonia
Lettonia	Estonia, Lituania
Polonia	Germania, Lituania

Tabella 4.2: Dominio del gioco nell'esempio minimax

Supponendo che sia MAX ad effettuare la prima mossa, si immagini che scelga la Lettonia.

Usando la funzione di utilità *naïve*, dove cioè i valori ammessi sono 1 e -1, l'albero di gioco avrebbe il seguente aspetto:

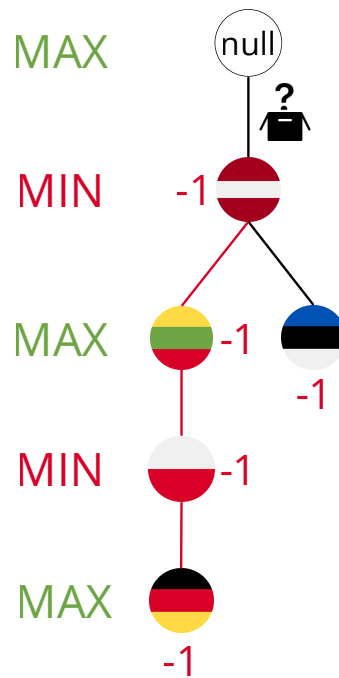


Figura 4.5: Albero di gioco a partire da Lettonia

Adesso, è il turno di *MIN*: ciò che si può immediatamente notare è che sia che scelga Lettonia, sia che scelga Estonia, vincerà certamente la partita. Per come è stato implementato l'algoritmo, infatti, a parità di valori di utilità (in questo caso -1), si sceglie il primo elemento così avvalorato. Quindi, la partita in questione si protrarrà per ben altre 3 mosse (*MIN* sceglie Lituania; *MAX* è costretto a selezionare Polonia; *MIN* può selezionare la sola Germania, lasciando senza mosse disponibili *MAX* e vincendo così la partita). Sarebbe stato molto più ragionevole che all'inizio *MIN* opzionasse l'Estonia, così da ottenere una vittoria immediata (in una sola mossa). Ecco perché questa versione della funzione di utilità è particolarmente "stupida", soprattutto se si considerano alberi di gioco con altezze non indifferenti.

La funzione di utilità adottata, invece, porta alla scelta del percorso più breve in caso di **vittoria garantita** e di quello più lungo in caso di **sconfitta inevitabile**. Quindi, considerando lo stesso esempio, i valori di utilità sarebbero i seguenti:

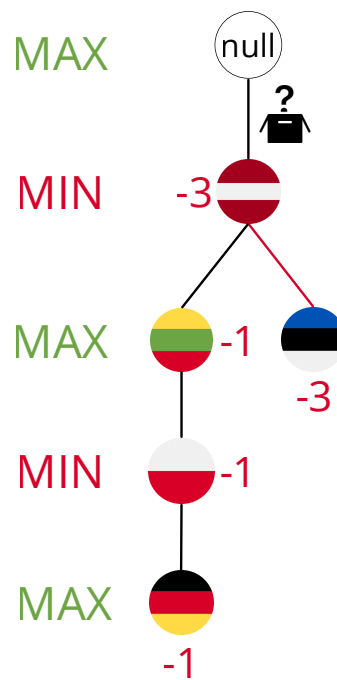


Figura 4.6: Albero di gioco a partire da Lituania

Siccome *MAX* ha selezionato la Lettonia⁵, il giocatore *MIN*, il cui obiettivo è minimizzare l'utilità, osserva che per la Lituania il valore è -1 mentre per l'Estonia è -3 . Poiché

$$\min\{-1, -3\} = -3,$$

MIN sceglierà l'Estonia (come previsto). Al contrario, se fosse stato il turno di *MAX* in una situazione simile (ovvero con mosse valutate tutte negativamente), esso avrebbe selezionato la Lituania, in quanto

$$\max\{-1, -3\} = -1.$$

In ogni caso, *MAX* avrebbe comunque perso, ma in questo modo lo avrebbe fatto "a testa alta", ossia prolungando la partita il più possibile.

4.2.6 Algoritmo Minimax euristico

Nel caso del Minimax euristico, la differenza importante rispetto ai due algoritmi precedentemente discussi sta nel fatto che la ricerca viene interrotta ad una profondità

⁵La prima nazione viene sempre generata in maniera casuale

prefissata, detta **profondità del taglio**. Siccome non è detto che alla profondità x tutti i nodi siano foglie, la funzione di utilità dev'essere sostituita da una funzione di valutazione $g(x)$ che stima il potenziale di un nodo intermedio dell'albero di gioco. Nel caso di **BorderGuessr**, si è deciso di definire la funzione di valutazione come segue:

- se si individua una foglia prima del taglio, $g(x) = f(x)$ (si va a calcolare essenzialmente l'utilità);
- se alla profondità x si ha a che fare con un nodo intermedio (quindi non terminale), semplicemente $g(x) = 0$

Questa formulazione garantisce che:

1. se ci sono mosse che portano certamente ad una vittoria, avranno la precedenza su quelle che portano ad un nodo intermedio;
2. se ci sono mosse che portano sicuramente ad una sconfitta, saranno considerate solo in assenza di mosse che conducano ad un nodo intermedio.

Infatti, l'unico modo per avere 0 come valore associato ad una certa mossa è che questa conduca ad un nodo intermedio oppure ad una foglia situata sul cammino di lunghezza pari al numero totale di Stati dalla radice. Nell'implementazione effettiva del gioco, la profondità del taglio è pari a 10 e la bontà di questa scelta verrà discussa nel capitolo successivo.

Volendo dare un'idea di che cosa significhi arrestare la ricerca, si può considerare il seguente esempio:

Con riferimento al dominio presentato nella **Tabella 4.1** e assumendo che la profondità del taglio sia 3, l'albero di gioco a partire dalla Bielorussia risulterà essere il seguente:

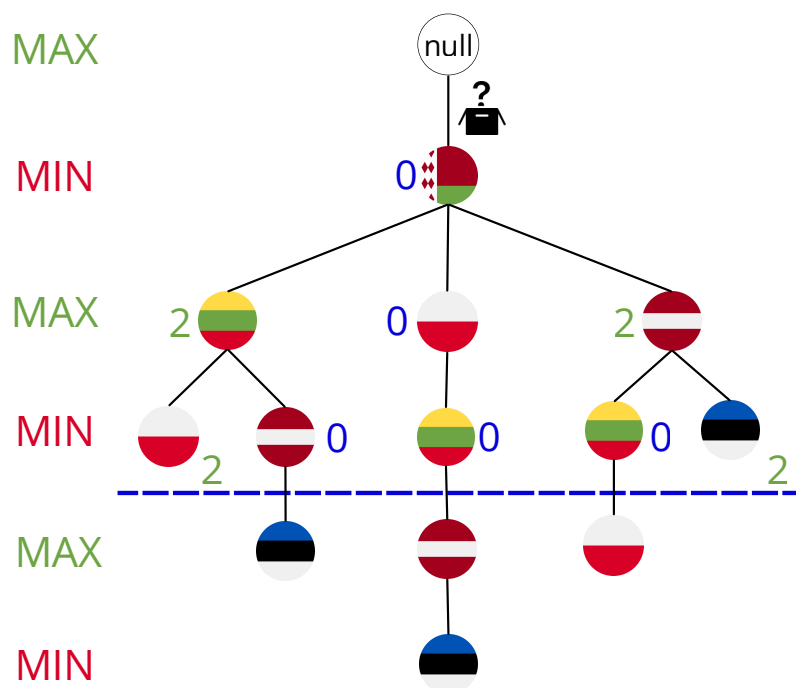


Figura 4.7: Albero di gioco a partire da Bielorussia

Con questa configurazione iniziale, applicando il Minimax euristico, *MIN* opterà la Polonia, siccome scegliendo Lituania o Lettonia andrò in contro ad una sconfitta se l'avversario gioca in maniera perfetta. Sebbene tale scelta non dia garanzie di vittoria (anche se in questo caso si può facilmente capire che la vittoria sarà certa), evita la sconfitta già al prossimo turno.

4.2.7 Monte Carlo tree search

A differenza degli algoritmi descritti finora, **MCTS** è di tipo probabilistico in quanto non esplora sistematicamente l'intero albero di ricerca, bensì effettua un certo numero di simulazioni casuali e poi, sulla base dei risultati ottenuti, sceglie la mossa da eseguire.

Come già ampiamente discusso nella **Sezione 2.1.11**, la Monte Carlo tree search si articola in quattro fasi, ripetute per un certo numero di iterazioni (nel caso di **BorderGuesser** sono pari a 1000) ogni qualvolta l'algoritmo è chiamato a decidere la prossima mossa. Altra osservazione importante da fare riguarda il fatto che MCTS costruisce un proprio albero su cui attua tali operazioni (che ha come radice il nodo cui corrisponde l'ultima mossa effettuata). Ciascun nodo di tale albero è corredato

da informazioni quali: genitore, figli, mossa cui corrisponde, visite ricevute, vittorie a cui ha portato la sua visita. Di seguito, per ciascuna di esse, verranno riportate le scelte implementative effettuate:

- **selezione:** in questa fase si effettua una sorta di DFS sull'albero "parallelo" che l'algoritmo mantiene fino ad arrivare ad una foglia. Questo processo è guidato da un'euristica, ovvero verrà selezionato il figlio che massimizza lo score UCB1;
- **espansione:** quando, durante la fase di selezione, si raggiunge un nodo che rappresenta uno stato non terminale ma che non è ancora stato esplorato, l'albero viene esteso aggiungendo uno o più figli corrispondenti a tutte le mosse possibili da quello stato (espansione completa⁶);
- **simulazione:** a partire dai nodi aggiunti con l'espansione, si sceglie il nodo n con numero più basso di potenziali figli (mosse legali)⁷ e si esegue un playout casuale fino ad uno stato terminale; l'esito sarà 0 se perde il giocatore che ha scelto n , 1 altrimenti;
- **retropropagazione:** l'esito viene propagato lungo tutto il cammino che porta da n alla radice avendo cura di aggiungere a tutti i nodi una visita in più e, soltanto ai nodi scelti dal giocatore trionfante, una vittoria in più.

Terminate le iterazioni, la mossa che MCTS restituisce è quella con il rapporto vittorie su visite più elevato. Esempio:

⁶Preferita rispetto a quella parziale in quanto il fattore di ramificazione è la massimo poco più di 10

⁷Stessa euristica usata nell'algoritmo Greedy

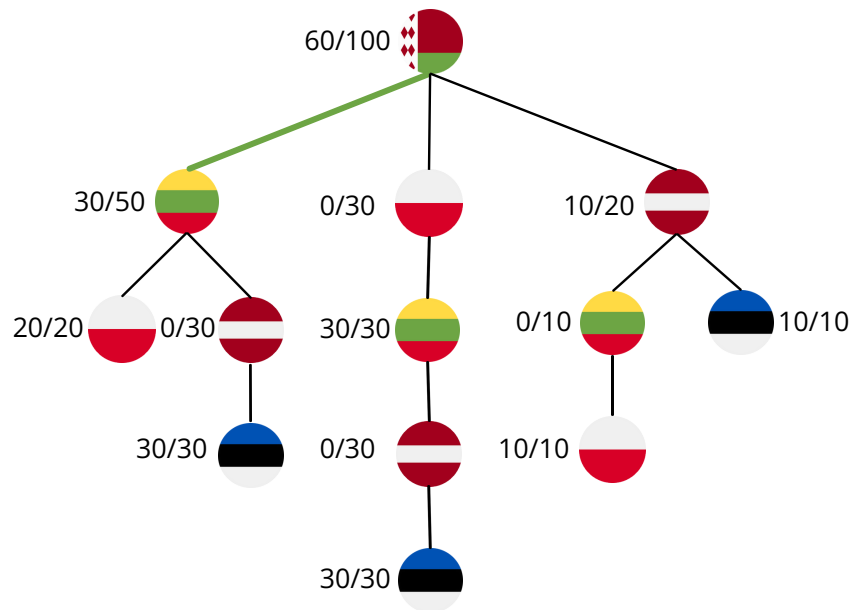


Figura 4.8: Albero MCTS a partire da Bielorussia

Assumendo che ad un certo punto della partita, il giocatore Z abbia scelto la Bielorussia. A questo punto, tocca al giocatore Y che lancia l'algoritmo MCTS. Quest'ultimo, sulla base di quanto detto prima, consiglierà come mossa quella di scegliere la Lituania in quanto ha un win rate pari a 0.6, a scapito di Polonia e Lettonia che hanno rispettivamente win rate pari a 0.0 (sconfitta aritmetica) e 0.5.

4.2.8 Algoritmo AlphaZero

L'algoritmo AlphaZero è l'unico, tra quelli analizzati, a fare uso di *machine learning*. In particolare, richiede l'impiego di una rete neurale che, una volta addestrata, è in grado di suggerire la prossima mossa con un certo margine di errore⁸. Poiché Kotlin non offre un supporto maturo per il *machine learning*, l'implementazione e l'addestramento della rete sono stati realizzati in Python. L'interazione con AlphaZero avviene tramite un'architettura client-server: un client basato su OkHTTP⁹ invia una richiesta al server FastAPI (concretamente parlando, nella richiesta si trova la board corrente), il quale esegue la rete neurale addestrata e restituisce al client la mossa consigliata da

⁸Il livello di accuratezza dipende strettamente dalla quantità e dalla qualità dei dati utilizzati per l'addestramento.

⁹Creato nel metodo `findBestMove()` della classe Kotlin AlphaZero

eseguire sotto forma di oggetto Move.

Quanto alla struttura della rete neurale, essa è costituita da tre layer lineari. Un layer si dice lineare se matrice dei pesi e vettore dei bias sono correlati alla matrice di input con operazioni matematiche lineari (somme algebriche e/o prodotti). Supponendo che il dominio del gioco sia costituito da n Stati, il numero di neuroni coinvolti è:

- $3 * n + 1$ per il primo layer; questo perché la rete neurale deve elaborare la board corrente come input. Poiché la board non può essere passata direttamente come oggetto, è necessario convertirla in un tensore, che rappresenta uno dei formati più noti accettati dalle NN¹⁰. Un tensore non è altro che un array multidimensionale di numeri. Un tensore di rango 0 è uno scalare, di rango 1 è un vettore, di rango 2 è una matrice e così via. Adesso, la domanda sorge spontanea: come trasformare delle stringhe in numeri? Occorre stabilire una codifica: in questo caso, si opterà per la **one-hot encoding** che rappresenta con un solo bit ciascun elemento (per evitare che la rete apprenda delle gerarchie fasulle). Ciò che deve necessariamente sapere la rete neurale è:

1. qual è lo Stato corrente (ultima mossa effettuata); codifica: impostare a 1 lo Stato corrente e a 0 tutti gli altri;
2. quali sono gli Stati già menzionati eccetto quello corrente (storico delle mosse); codifica: impostare ad 1 gli Stati già menzionati e a 0 tutti gli altri;
3. quali sono gli Stati che possono essere scelti come mossa successiva; codifica: impostare ad 1 gli Stati raggiungibili con una mossa legale da quello corrente, a 0 tutti gli altri;
4. chi detiene il turno; codifica: 1 se è il turno di player1, 0 altrimenti.

Ciascuno di questi elementi prende il nome di **feature**. Purtroppo non finisce qui: quello che si ottiene infatti è un tensore di rango 1 (vettore) mentre le reti neurali si aspettano gli input in formato [batchSize, numFeatures]¹¹ (tensore di rango 2). Per aggiungere un ulteriore dimensione si può usare il metodo `unsqueeze(0)`. Così facendo, alla rete viene passata una matrice $[1, 3 * n + 1]$,

¹⁰Neural Network

¹¹La `batchSize` indica il numero di esemplari forniti in input; in questo caso, 1

ovvero "un esempio costituito da $3 * n + 1$ feature".

Esempio per $n=5$ (considerando il dominio riportato nella **Tabella 4.1**):

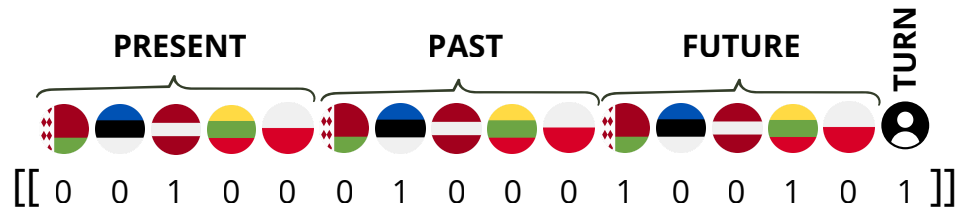


Figura 4.9: Tensore di input [1, 16]

- 256 per il secondo layer; questa scelta ha principalmente una motivazione teorica: le potenze di 2 ottimizzano bene i calcoli sulle GPU (comunque sia, la regola da seguire è non usare troppi o troppo pochi neuroni rispetto a quelli del primo layer per evitare di incorrere in problemi di under/overfitting);
- $n + 1$ per il terzo layer. La rete neurale è strutturata con due teste di output (*multi-head architecture*):
 1. la prima testa fornisce, per ciascuna mossa legale, la probabilità che quella scelta conduca alla vittoria (policy head);
 2. la seconda testa stima la probabilità complessiva di vittoria nel match a partire dalla posizione corrente (value head).

Occorre fare una piccola precisazione sulla policy head: la rete neurale restituisce un vettore logits, che possono assumere anche valori negativi. Di conseguenza, è necessario fare una prima scrematura di tali valori ponendo a $-\infty$ quelli associati a mosse illegali (usando una maschera). Dopodiché, per ottenere una distribuzione di probabilità sulle mosse legali, questi valori vengono trasformati tramite la funzione **softmax** (che non considera per default

quelli posti a $-\infty$).

Esempio di possibile output (già ripulito) per $n=5$, avendo dato in input il tensore in **Figura 4.9**:

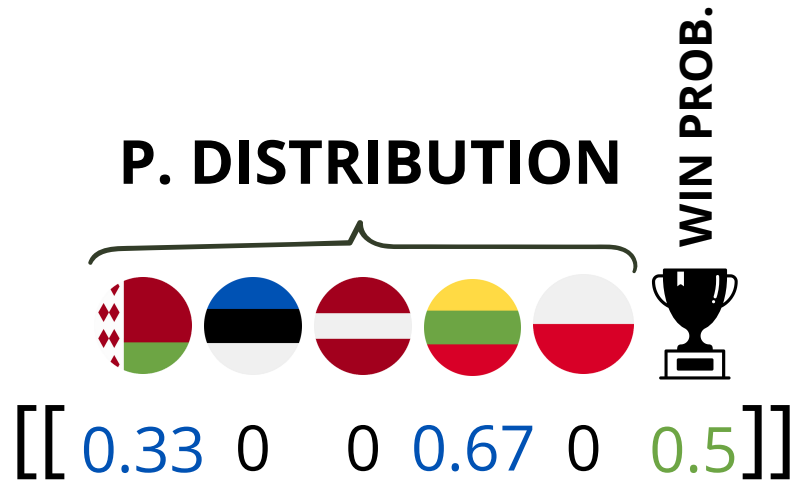


Figura 4.10: Tensore di output $[1, 6]$

Adesso che è chiaro cosa prende in input la rete neurale e cosa produce in output, resta da chiarire cosa avviene nel mezzo, cioè durante la fase di training. Il processo di addestramento si svolge in modo iterativo, seguendo questi passaggi principali:

- generazione dati: per ciascun ciclo di training (o *iteration*), si giocano m partite di auto-play utilizzando la Monte Carlo Tree Search (MCTS)¹², con un numero fisso di simulazioni i per mossa;
- valutazione: al termine del ciclo, il modello appena addestrato viene confrontato in p partite contro il modello considerato “migliore” fino a quel momento (che coincide con se stesso nel caso del primo ciclo);
- selezione del modello: se il nuovo modello ottiene un win rate almeno pari a 0.55, viene promosso a nuovo “miglior modello” e salvato.
- aggiornamento dei pesi: i dati di training generati vengono raccolti e usati per aggiornare la rete per e epoche tramite ottimizzazione dei parametri.

¹²Leggermente diverso dalla versione implementata in Kotlin in quanto nelle simulazioni viene scelto lo Stato che ha il più basso numero di vicini “consentiti”, ovvero non ancora menzionati; il nodo che verrà espanso invece viene scelto direttamente dalla rete neurale

Un aspetto fondamentale è che, ad ogni nuovo ciclo di training, il modello non riparte da zero, ma continua ad apprendere a partire dai pesi ottenuti nei cicli precedenti. In questo modo la rete “ricorda” l’esperienza accumulata nelle iterazioni passate, raffinandola progressivamente. Ciò è possibile salvando i pesi della rete e lo stato dell’ottimizzazione per ogni ciclo (anche se si rivela fallimentare). L’algoritmo usato per l’ottimizzazione è *Adam* che combina momentum e scaling adattivo, permettendo un apprendimento rapido e stabile anche in presenza di gradienti irregolari. Esempio della sua efficacia considerando 10 epoche:

```
Training nuovo modello...
Epoch 1/10 - Loss: 1.824038241 (Policy: 41.116619349, Value: 1.463273403)
Epoch 2/10 - Loss: 0.703434958 (Policy: 16.234851062, Value: 0.185958859)
Epoch 3/10 - Loss: 0.486818177 (Policy: 11.342512995, Value: 0.021648717)
Epoch 4/10 - Loss: 0.302131788 (Policy: 7.012047201, Value: 0.040841705)
Epoch 5/10 - Loss: 0.202981884 (Policy: 4.727762870, Value: 0.010595485)
Epoch 6/10 - Loss: 0.150497447 (Policy: 3.500518430, Value: 0.012656366)
Epoch 7/10 - Loss: 0.115449798 (Policy: 2.686789613, Value: 0.008241599)
Epoch 8/10 - Loss: 0.080177946 (Policy: 1.844766069, Value: 0.026887840)
Epoch 9/10 - Loss: 0.062622550 (Policy: 1.457503874, Value: 0.004341257)
Epoch 10/10 - Loss: 0.049544852 (Policy: 1.145407157, Value: 0.011155470)
Valutazione torneo...
```

Figura 4.11: Ottimizzazione usando algoritmo Adam

Come si può notare, nel descrivere l’addestramento di AlphaZero non sono stati forniti numeri concreti: questo aspetto verrà discusso nel capitolo successivo.

Riassumendo, senza entrare troppo nei dettagli, l’output della rete deriva da operazioni di algebra lineare, in particolare moltiplicazioni tra matrici (input e pesi) e aggiunta dei bias; il risultato viene poi passato al layer successivo attraverso funzioni di attivazione non lineari¹³. Durante l’addestramento, ad ogni epoca, si applica inoltre la **backward propagation**: l’errore calcolato all’uscita da un layer viene propagato

¹³In questo specifico caso, si è fatto uso di ReLU (che restituisce 0 se l’input è negativo, altrimenti restituisce l’input stesso) e Tanh (che schiaccia l’output tra -1 e 1)

all'indietro e usato per aggiornare i pesi in modo da migliorare progressivamente le previsioni.

4.3 IMPLEMENTAZIONE PER L'APP MOBILE

4.3.1 Ambiente di sviluppo ed architettura

Siccome l'obiettivo è quello di pubblicare **BorderGuessr** su Play Store (in un futuro non troppo lontano), si è scelto di adottare *Android Studio* come ambiente di sviluppo integrato (IDE) e *Kotlin* come linguaggio principale sia per la parte di back-end che quella grafica. Questa scelta è motivata dalla piena compatibilità di Kotlin con l'ecosistema Android e dal supporto ufficiale garantito da Google. Android Studio, inoltre, offre strumenti avanzati per il design dell'interfaccia utente, il debug e l'ottimizzazione delle prestazioni, rendendolo la piattaforma ideale per portare il progetto dalla fase di prototipazione fino alla eventuale pubblicazione ufficiale. In aggiunta, per lo sviluppo dell'interfaccia grafica (UI) è stato scelto di adottare *Jetpack Compose* al posto del tradizionale sistema basato su *Android View*. Compose, infatti, permette di descrivere l'UI direttamente in Kotlin, evitando di mescolare il codice XML delle *Android View* con il codice Kotlin del back-end.

Per facilitare l'eventuale manutenzione dell'applicazione, nonché la comprensione della struttura della stessa, si è pensato di svilupparla seguendo il modello **MVVM** (Model-View-View Model). Nello specifico:

- nel **Model** ricadono i dati e la logica di business;
- nella **View**, invece, è possibile trovare le varie interfacce utente;
- nel **View Model**, infine, funge da ponte tra i due layer precedentemente elencati in quanto si occupa di procurarsi i dati dal **Model** e di fornirli belli e pronti alla **View**.

4.3.2 Target di utenti e modalità di gioco

Aldilà degli aspetti puramente tecnici ed implementativi dell'applicativo che s'intende realizzare, che non sono, tra l'altro, l'argomento principale di questo lavoro

di tesi, è bene definire quello che è il target di utenti. Infatti **BorderGuesser** è stato pensato sia per coloro che si avvicinano al mondo della geografia politica, sia per coloro che invece desiderano testare le proprie conoscenze ed eventualmente migliorarle. A tal proposito, si è deciso di fornire quattro distinti livelli di difficoltà:

- **principiante:** l'idea qui è di fare in modo che l'agente intelligente vinca soltanto quando sia costretto a farlo, cioè non ha altre vie d'uscita. Ciò infatti permetterebbe di favorire l'apprendimento dell'utente in quanto le partite durerebbero di più (a patto che quest'ultimo fornisca sempre una mossa valida). L'algoritmo migliore in grado di riuscire in quest'intento è di tipo Greedy e usa la seguente euristica: selezionare il paese che ha il numero più alto possibile di vicini non ancora menzionati nel corso del game corrente;
- **medio:** sulla base dei risultati dell'analisi empirica condotta nel **Capitolo 5**, risulta avere una efficacia intermedia un altro algoritmo Greedy, la cui euristica è spiegata nella **Sezione 4.2.4**;
- **esperto:** a questo stadio di difficoltà, si potrebbe pensare di far giocare l'utente contro il Minimax standard o qualche sua versione ottimizzata. Il problema però è sempre e soltanto uno: il tempo. Infatti, attendere già solo qualche secondo prima che l'agente intelligente compia la propria mossa rischia di far perdere l'interesse nel giocatore umano. Ragion per cui, l'algoritmo che risulta conciliare al meglio tempo di risposta e qualità delle mosse individuate, risulta essere il Minimax euristico (ovviamente con una profondità di taglio opportuna, in questo caso, pari a 8 per questioni di performance).

4.3.3 Navigational Path

Senza entrare troppo nei dettagli implementativi, di seguito verrà illustrato il percorso di navigazione che porta dalla schermata principale dell'applicazione, passando per il game vero e proprio, fino alla schermata di fine partita.

Nome schermata	Spiegazione	Schermate raggiungibili
home	schermata di avvio dell'applicazione	level, premendo "Gioca"
level	scelta del livello di difficoltà	mapSelection, indipendentemente dal livello scelto
mapSelection	scelta della mappa con cui si desidera giocare	game, indipendentemente dalla mappa scelta
game	schermata di gioco dove in alto c'è la lista degli Stati già detti mentre al centro quello corrente. In basso, invece, il campo in cui inserire la nazione successiva	game, se lo Stato inserito non porta alla conclusione della partita; result, se la partita termina
result	schermata che segnala la vittoria o la sconfitta dell'utente	level, premendo "Menù"; game, premendo "Nuova partita"

Tabella 4.3: Elenco delle View dell'app BorderGuessr

Per dare un'idea di come funzioni il tutto, di seguito una rappresentazione grafica del navigational path:

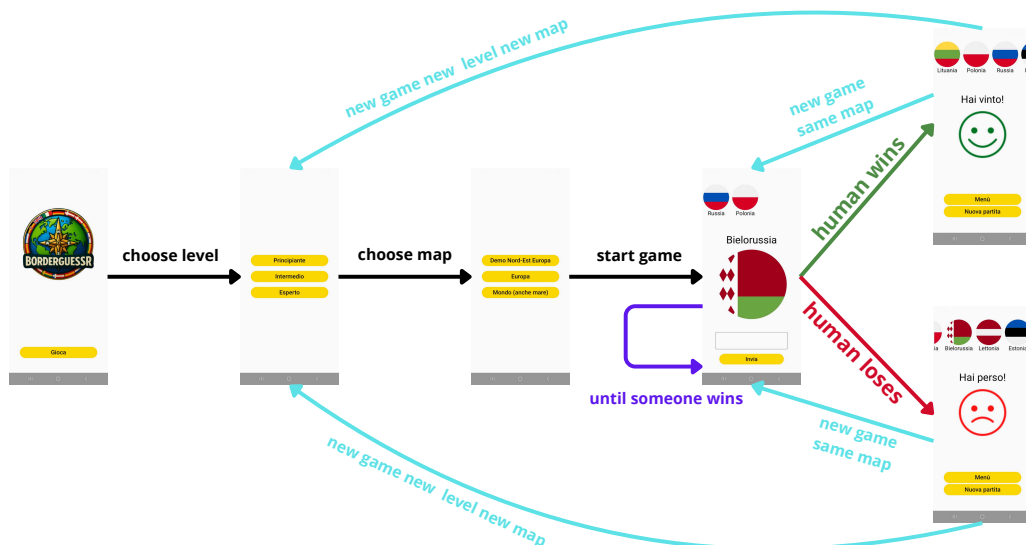


Figura 4.12: Navigational path dell'app BorderGuessr

CAPITOLO 5

Analisi empirica

In questo capitolo si tenterà di effettuare una analisi empirica dei vari algoritmi implementati per giocare a **BorderGuessr**, con l'obiettivo di valutarne i punti di forza e quelli di debolezza.

5.1 Introduzione

Condurre un'analisi empirica significa studiare qualcosa basandosi su dati concreti, frutto dell'osservazione/esperienza diretta, piuttosto che rifacendosi a teorie, idee, ipotesi. L'esempio forse più conosciuto di analisi empirica è la legge di Moore, formulata da Gordon Moore nel 1965. Egli osservò che il numero di transistor presenti su un microprocessore raddoppiava approssimativamente ogni due anni, aumentando così la capacità di calcolo dei computer [14].

Sulla base di quanto appena detto, uno dei due scopi di questo lavoro di tesi¹ è tentare di effettuare un'analisi empirica degli algoritmi usati nei giochi a somma zero con informazione perfetta nel contesto di un gioco in particolare: **BorderGuessr**.

Di conseguenza, dopo una breve descrizione delle modalità di conduzione dell'analisi

¹L'altro è offrire una soluzione per contrastare l'analfabetismo geografico

e dell'hardware usato, verranno riportati i risultati ottenuti ed infine, per ciascun algoritmo, si analizzeranno eventuali punti di forza e/o debolezza.

5.2 Modalità conduzione analisi

Nel momento in cui si decide di condurre un'analisi empirica, occorre decidere come effettuarla e cosa si ritiene opportuno osservare ai fini della stessa. Per quanto riguarda il **come**, poiché un **game** comincia con uno Stato a caso, se si hanno a disposizione n Stati, ha senso far sfidare due algoritmi per volta in n **game**, ciascuna con uno Stato iniziale differente. L'insieme di questi n **game** prende il nome di **match**. Inoltre, per evitare che chi effettui la prima mossa abbia un qualsiasi tipo di vantaggio, sono stati adottati i seguenti accorgimenti:

- se gli Stati sono n , sono previsti due **match**: nel primo lo Stato scelto casualmente viene attribuito all'algoritmo X , nel secondo, invece, all'algoritmo Y (in gergo calcistico, è una sorta di andata e ritorno);
- il **game** più breve in assoluto deve comunque prevedere tre mosse: la prima, essendo frutto del caso, non misura la bontà dell'algoritmo cui viene attribuita, ragion per cui quest'ultimo deve avere la possibilità di effettuare una mossa frutto della sua effettiva elaborazione, dopo che l'avversario abbia compiuto la propria. Di conseguenza, si è reso necessario apportare delle modifiche ad alcuni Stati: se questi confinano con un'enclave², tale enclave non va inserita tra i confini. Ad esempio, l'Italia ha ben 2 enclavi: San Marino e Città del Vaticano, che figureranno nella lista degli Stati ma non in quella dei confini dell'Italia. Casi simili si possono riscontrare anche con Francia e Principato di Monaco, Sudafrica e Lesotho, ... Ciò quindi permette di usare le enclavi come Stati iniziale, ma non come mosse di gioco.

Di conseguenza, si è deciso di far sfidare tutti gli algoritmi fra loro, in una specie di campionato con l'assegnazione di 3 punti a vittoria, 1 in caso di pareggio e 0 in caso di sconfitta. Per quanto concerne il **cosa** osservare, l'interesse è ricaduto su numero di game vinti, tempo medio di risposta (misurato in ns) per game e per match e infine,

²Stato completamente circondato dal territorio di un altro Stato

chiaramente, posizione in classifica. In caso di arrivo a pari punti, viene piazzato in una posizione migliore l'algoritmo che ha vinto il maggior numero di game. In caso di ulteriore parità, a decidere il posizionamento finale sarà il tempo medio impiegato per effettuare una mossa.

Siccome l'algoritmo Minimax nella sua versione standard e l'algoritmo Alpha-Beta sono praticamente inutilizzabili per alberi di ricerca con elevate profondità e fattore di ramificazione, è sembrato ragionevole organizzare ben due campionati:

1. il primo vede la partecipazione di ben sei algoritmi, usando un dominio di soli 34 Stati (maggiori dettagli nella sezione 5.4);
2. il secondo vede la partecipazione di soli quattro algoritmi, usando però un dominio di 196 Stati (maggiori dettagli nella sezione 5.5).

Tutte le informazioni relative ad un *game* sono state raccolte in un oggetto *GameAnalysis* contenente informazioni quali:

- Stato iniziale;
- elenco Stati selezionati nel corso della partita;
- giocatore 1 (cui è attribuita la mossa della scelta dello Stato iniziale);
- giocatore 2;
- tempo medio per effettuare una mossa da parte del giocatore 1 (in *ns*);
- tempo medio per effettuare una mossa da parte del giocatore 2 (in *ns*);
- vincitore (true se giocatore 1, false altrimenti).

Per questo motivo, è stato possibile produrre un report per ogni **match**, consultabili qui. Esempio contenuto di un report:

```

Si sono sfidati: AlphaZero - Alphabeta Pruning
La partita è iniziata con Serbia
ed è proseguita con: Romania - Ungheria - Slovacchia - Cechia - Polonia - Germania - Austria -
Italia - Slovenia - Croazia - Bosnia ed Erzegovina
Per calcolare una mossa, AlphaZero ha impiegato mediamente 41521166 ns, mentre Alphabeta Pruning ha
impiegato mediamente 4765100 ns
La partita si è conclusa con una vittoria di Alphabeta Pruning in 11 mosse

Si sono sfidati: AlphaZero - Alphabeta Pruning
La partita è iniziata con Svizzera
ed è proseguita con: Francia - Italia - Slovenia - Croazia - Bosnia ed Erzegovina - Serbia -
Romania - Ungheria - Slovacchia - Cechia - Polonia - Germania - Paesi Bassi - Belgio - Lussemburgo
Per calcolare una mossa, AlphaZero ha impiegato mediamente 38805875 ns, mentre Alphabeta Pruning ha
impiegato mediamente 4036900 ns
La partita si è conclusa con una vittoria di Alphabeta Pruning in 15 mosse

Punteggio finale: 0-34
Average time AlphaZero: 48732244 ns
Average time AlphabetaPruning: 11087789 ns

```

Figura 5.1: Contenuto di un report

5.3 Ambiente di esecuzione

Le analisi sono state eseguite su un PC portatile (quello dello scrivente) con le seguenti caratteristiche:

- processore Intel Core i7 di 12^a generazione, modello 1255U, con velocità di base di 1,70 GHz e capacità di eseguire più attività contemporaneamente grazie ai 10 core, di cui:
 - 2 ad alte prestazioni, aventi 2 thread ciascuno (per un totale di 4 thread);
 - 8 a basso consumo, ciascuno avente un solo thread;
- 16 GB di RAM;
- scheda video integrata Intel Iris Xe con 8182 MB di memoria così suddivisi:
 - 128 MB di memoria video dedicata;
 - 8054 MB di memoria condivisa con la RAM;
- sistema operativo Windows 11 a 64 bit.

Inoltre, si è fatto uso dei seguenti componenti software:

- IntelliJ IDEA 2025.2 (Ultimate Edition);
- Kotlin versione 252.23892.409-IJ (integrato in IntelliJ IDEA);

- `kotlinx-serialization-json` versione 1.6.3, per la serializzazione/deserializzazione di dati JSON;
- `OkHttp` versione 5.1.0, per la gestione delle richieste HTTP;
- `Gradle` bundled 252.23892.409, come software di building;
- `PyCharm` 2024.2.4, come IDE per la parte di codice in Python (`AlphaZero`);
- Python versione 3.13.0;
- `FastAPI` versione 0.116.1, utilizzata per creare API web ad alte prestazioni, con le quali il client Kotlin comunica tramite `OkHttpClient`;
- `PyTorch` (`torch`) versione 2.8.0, libreria Python usata per implementare e addestrare la rete neurale dell'algoritmo `AlphaZero`.

La configurazione completa garantisce che i risultati ottenuti siano riproducibili su macchine con caratteristiche analoghe. Eventuali differenze hardware, come numero di core o frequenza della CPU, possono influenzare i tempi di esecuzione, ma non la correttezza degli algoritmi.

5.4 Campionato 1

Hanno preso parte a questo campionato i seguenti algoritmi:

- **Minimax standard**;
- **Alpha-Beta pruning**;
- **Minimax euristico**, con profondità del taglio fissata a 10;
- **Monte Carlo tree search**;
- **AlphaZero**, la cui rete neurale è stata addestrata con i seguenti parametri:
 1. 10 cicli;
 2. 500 game per ciclo;
 3. 150 iterazioni MCTS per mossa;
 4. 34 game di valutazione;
 5. 15 epoche;

6. batch size pari a 32;

tempo stimato per l'addestramento: circa 3 ore;

Quanto al dominio, esso era costituito da sole 34 nazioni (del Vecchio Continente) e sono stati presi in considerazione i soli confini terrestri. Alcune precisazioni in merito: l'Eurotunnel non è da considerarsi come confine terrestre tra Francia e Regno Unito; il ponte di Öresund non è da considerarsi come confine terrestre tra Danimarca e Svezia.

5.4.1 Partite

Di seguito le varie giornate di campionato; l'ordine delle stesse è totalmente casuale e influente ai fini dell'analisi empirica.

I giornata

Casa	Risultato	Trasferta
GreedySearch tm: 2,612 μ s	0 - 34	HeuristicMinimax tm: 1211,686 μ s
MonteCarloTreeSearch tm: 3215,158 μ s	12 - 22	AlphaZero tm: 91582,810 μ s
AlphaBetaPruning tm: 3298,192 μ s	2 - 32	StandardMinimax tm: 667557,500 μ s

Tabella 5.1: I giornata campionato 1

II giornata

Casa	Risultato	Trasferta
GreedySearch tm: 2,038 μ s	1 - 33	StandardMinimax tm: 1887890,948 μ s
HeuristicMinimax tm: 866,778 μ s	33 - 1	MonteCarloTreeSearch tm: 4595,061 μ s
AlphaBetaPruning tm: 6751,108 μ s	30 - 4	AlphaZero tm: 83757,331 μ s

Tabella 5.2: II giornata campionato 1

III giornata

Casa	Risultato	Trasferta
GreedySearch tm: 1,719 μs	1 - 33	AlphaBetaPruning tm: 17807,091 μs
HeuristicMinimax tm: 872,974 μs	31 - 3	AlphaZero tm: 90716,911 μs
StandardMinimax tm: 542158,460 μs	31 - 3	MonteCarloTreeSearch tm: 4510,815 μs

Tabella 5.3: III giornata campionato 1

IV giornata

Casa	Risultato	Trasferta
GreedySearch tm: 2,215 μs	11 - 23	MonteCarloTreeSearch tm: 5677,670 μs
HeuristicMinimax tm: 453,102 μs	2 - 32	AlphaBetaPruning tm: 7138,795 μs
StandardMinimax tm: 416434,391 μs	30 - 4	AlphaZero tm: 81514,791 μs

Tabella 5.4: IV giornata campionato 1

V giornata

Casa	Risultato	Trasferta
GreedySearch tm: 10,469 μs	14 - 20	AlphaZero tm: 102818,251 μs
HeuristicMinimax tm: 416,029 μs	2 - 32	StandardMinimax tm: 688825,479 μs
MonteCarloTreeSearch tm: 2316,495 μs	0 - 34	AlphaBetaPruning tm: 17393,692 μs

Tabella 5.5: V giornata campionato 1

VI giornata

Casa	Risultato	Trasferta
StandardMinimax tm: 372125,026 μs	29 - 5	GreedySearch tm: 3,429 μs
MonteCarloTreeSearch tm: 2657,276 μs	0 - 34	HeuristicMinimax tm: 1303,048 μs
AlphaZero tm: 48732,244 μs	0 - 34	AlphaBetaPruning tm: 11087,789 μs

Tabella 5.6: VI giornata campionato 1

VII giornata

Casa	Risultato	Trasferta
HeuristicMinimax tm: 869,773 μs	32 - 2	GreedySearch tm: 3,501 μs
AlphaZero tm: 54986,250 μs	23 - 11	MonteCarloTreeSearch tm: 4710,418 μs
StandardMinimax tm: 178051,570 μs	2 - 32	AlphaBetaPruning tm: 6099,215 μs

Tabella 5.7: VII giornata campionato 1

VIII giornata

Casa	Risultato	Trasferta
AlphaBetaPruning tm: 6757,670 μs	29 - 5	GreedySearch tm: 3,125 μs
AlphaZero tm: 54629,580 μs	0 - 34	HeuristicMinimax tm: 1356,021 μs
MonteCarloTreeSearch tm: 2198,501 μs	0 - 34	StandardMinimax tm: 1737121,078 μs

Tabella 5.8: VIII giornata campionato 1

IX giornata

Casa	Risultato	Trasferta
MonteCarloTreeSearch tm: 4108,130 μs	29 - 5	GreedySearch tm: 4,474 μs
AlphaBetaPruning tm: 3680,208 μs	2 - 32	HeuristicMinimax tm: 639,757 μs
AlphaZero tm: 45237,828 μs	0 - 34	StandardMinimax tm: 1114111,928 μs

Tabella 5.9: IX giornata campionato 1**X giornata**

Casa	Risultato	Trasferta
AlphaZero tm: 67510,527 μs	23 - 11	GreedySearch tm: 13,456 μs
StandardMinimax tm: 194791,355 μs	2 - 32	HeuristicMinimax tm: 642,234 μs
AlphaBetaPruning tm: 9259,845 μs	31 - 3	MonteCarloTreeSearch tm: 4452,910 μs

Tabella 5.10: X giornata campionato 1**5.4.2 Classifica finale**

Pos.	Algoritmo	P.ti	G	V	Pa	Pe	GV	GP	T.medio (μs)
1	HeuristicMinimax	24	10	8	0	2	266	74	863,140
2	AlphaBetaPruning	24	10	8	0	2	259	81	8927,360
3	StandardMinimax	24	10	8	0	2	259	81	779906,773
4	AlphaZero	12	10	4	0	6	99	241	144297,305
5	MonteCarloTreeSearch	6	10	2	0	8	82	258	3844,243
6	GreedySearch	0	10	0	0	10	55	285	4,704

Tabella 5.11: Classifica finale campionato 1**Legenda:**

- **P.ti**: punti conquistati;
- **G**: match giocati;
- **V**: numero di vittorie;
- **Pa**: numero di pareggi;
- **Pe**: numero di sconfitte;
- **GV**: numero di game vinti;
- **GP**: numero di game persi;
- **T.medio**: tempo medio per effettuare una singola mossa (scegliere il prossimo Stato), misurato in μs (microsecondi).

5.5 Campionato 2

Hanno preso parte a questo campionato i seguenti algoritmi:

- **Minimax euristico**, con profondità del taglio fissata a 10;
- **Monte Carlo tree search**;
- **AlphaZero**, la cui rete neurale è stata addestrata con i seguenti parametri:
 1. 10 cicli;
 2. 200 game per ciclo;
 3. 30 iterazioni MCTS per mossa;
 4. 30 game di valutazione;
 5. 15 epoche;
 6. batch size pari a 32;

tempo stimato per l'addestramento: circa 12 ore;

Quanto al dominio, esso era costituito da ben 196 nazioni (per conoscere quali, si rimanda alla sezione 4.1.4) e sono stati presi in considerazione sia i confini terrestri che marittimi (altrimenti sarebbe stato necessario escludere gli Stati insulari)³

³La lista completa, con annessa spiegazione di cosa si intende per confine terrestre e confine marittimo, è consultabile al seguente link: https://en.wikipedia.org/wiki/List_of_countries_and_territories_by_land_and_maritime_borders

5.5.1 Partite

Di seguito le varie giornate di campionato; l'ordine delle stesse è totalmente casuale e ininfluyente ai fini dell'analisi empirica.

I giornata

Casa	Risultato	Trasferta
GreedySearch tm: 7,817 μ s	24 - 172	HeuristicMinimax tm: 109457,640 μ s
MonteCarloTreeSearch tm: 61741,038 μ s	36 - 160	AlphaZero tm: 2529840,686 μ s

Tabella 5.12: I giornata campionato 2

II giornata

Casa	Risultato	Trasferta
GreedySearch tm: 48,949 μ s	173 - 23	AlphaZero tm: 3640497,415 μ s
HeuristicMinimax tm: 77402,976 μ s	196 - 0	MonteCarloTreeSearch tm: 79963,144 μ s

Tabella 5.13: II giornata campionato 2

III giornata

Casa	Risultato	Trasferta
GreedySearch tm: 7,491 μ s	176 - 20	MonteCarloTreeSearch tm: 85598,501 μ s
HeuristicMinimax tm: 79385,507 μ s	196 - 0	AlphaZero tm: 1452578,409 μ s

Tabella 5.14: III giornata campionato 2

IV giornata

Casa	Risultato	Trasferta
HeuristicMinimax tm: 57745,184 μs	164 - 32	GreedySearch tm: 7,707 μs
AlphaZero tm: 2916737,518 μs	165 - 31	MonteCarloTreeSearch tm: 82934,733 μs

Tabella 5.15: IV giornata campionato 2

V giornata

Casa	Risultato	Trasferta
AlphaZero tm: 1985592,628 μs	17 - 179	GreedySearch tm: 87,302 μs
MonteCarloTreeSearch tm: 57249,965 μs	0 - 196	HeuristicMinimax tm: 107941,510 μs

Tabella 5.16: V giornata campionato 2

VI giornata

Casa	Risultato	Trasferta
MonteCarloTreeSearch tm: 68999,258 μs	18 - 178	GreedySearch tm: 10,594 μs
AlphaZero tm: 2355016,976 μs	0 - 196	HeuristicMinimax tm: 288907,565 μs

Tabella 5.17: VI giornata campionato 2

5.5.2 Classifica finale

Pos.	Algoritmo	P.ti	G	V	Pa	Pe	GV	GP	T.medio (μs)
1	HeuristicMinimax	18	6	6	0	0	1120	56	120140,064
2	GreedySearch	12	6	4	0	2	762	414	28,310
3	AlphaZero	6	6	2	0	4	365	811	2480043,938
4	MonteCarloTreeSearch	0	6	0	0	6	105	1071	72747,773

Tabella 5.18: Classifica finale campionato 2

Legenda:

- **P.ti:** punti conquistati;
- **G:** match giocati;
- **V:** numero di vittorie;
- **Pa:** numero di pareggi;
- **Pe:** numero di sconfitte;
- **GV:** numero di game vinti;
- **GP:** numero di game persi;
- **T.medio:** tempo medio per effettuare una singola mossa (scegliere il prossimo Stato), misurato in μs (microsecondi).

5.6 Osservazioni

5.6.1 Campionato 1

Sulla base dei risultati e della classifica finali del campionato 1, è possibile fare le seguenti considerazioni, che mettono in luce aspetti teorici con quelli effettivamente osservati nella pratica.

StandardMinimax, AlphaBetaPruning e HeuristicMinimax

Dando uno sguardo a quella che è la classifica finale, non sorprende affatto che StandardMinimax, AlphaBetaPruning e HeuristicMinimax siano in testa a pari punti.

Questo perché si tratta essenzialmente di giocatori perfetti; pertanto, non era in discussione la loro leadership. Negli scontri diretti, ad avere la meglio è sempre stato l'algoritmo che muoveva per secondo, vincendo ogni volta per 32 a 2. Sebbene fosse lecito aspettarsi un risultato del genere per StandardMinimax e AlphaBetaPruning (poiché difatti si tratta dello stesso ed identico algoritmo), al contrario, non era per nulla scontato che HeuristicMinimax reggesse così bene il confronto con due avversari teoricamente più forti. L'unica spiegazione sensata che può venire in mente è che la profondità di taglio (pari a 10) non abbia in alcun modo limitato le capacità predittive del Minimax euristico. In altre parole, guardare in avanti di 10 mosse, è più che sufficiente nel dominio considerato. Le uniche volte in cui l'algoritmo che muoveva per primo è riuscito a vincere sono state quando lo Stato iniziale era la Danimarca oppure il Portogallo. Analizzando meglio i report relativi a questi tre algoritmi, vien fuori che il Minimax euristico quando vince, vince con difficoltà (impiegando più mosse rispetto alla vittoria che riporta l'avversario a parti invertite); quando perde, perde con maggiore facilità (impiegando meno mosse rispetto alla sconfitta che riporta l'avversario a parti invertite). Ciò è sicuramente imputabile alla profondità del taglio, che resta comunque subottimale.

Sebbene negli scontri diretti regni la totale parità tra Minimax standard, Alpha-Beta e Minimax euristico, la cosa sorprendente è che nei confronti con gli altri algoritmi (MCTS, Greedy e AlphaZero), il Minimax euristico sia riuscito a vincere un maggior numero di game (202 contro 195). In particolare, ne ha vinto uno in più (in casa) contro AlphaZero, due in più contro Monte Carlo Tree Search (in casa) e quattro (3 in casa e 1 in trasferta) in più contro l'algoritmo Greedy. Questo non implica che il Minimax euristico sia superiore in senso assoluto rispetto al Minimax standard o alla potatura Alpha-Beta, ma solo che, contro avversari non perfetti, il diverso stile di gioco prodotto dall'euristica può esporre più facilmente le loro debolezze.

Esempio:

Si sono sfidati: Alphabeta Pruning - Greedy
 La partita è iniziata con Paesi Bassi
 ed è proseguita con: Belgio - Lussemburgo - Francia - Italia - Slovenia - Croazia - Bosnia ed
 Erzegovina - Serbia - Romania - Ungheria - Slovacchia - Austria - Liechtenstein - Svizzera -
 Germania - Cechia - Polonia - Lituania - Bielorussia - Lettonia - Estonia - Russia - Norvegia -
 Svezia - Finlandia
 Per calcolare una mossa, Alphabeta Pruning ha impiegato mediamente 4671315 ns, mentre Greedy ha
 impiegato mediamente 2566 ns
 La partita si è conclusa con una vittoria di Greedy in 25 mosse

Figura 5.2: report AlphaBetaPruning vs GreedySearch partendo dai Paesi Bassi

Si sono sfidati: Heuristic Minimax - Greedy
 La partita è iniziata con Paesi Bassi
 ed è proseguita con: Belgio - Germania - Lussemburgo - Francia - Spagna - Andorra
 Per calcolare una mossa, Heuristic Minimax ha impiegato mediamente 875433 ns, mentre Greedy ha
 impiegato mediamente 2733 ns
 La partita si è conclusa con una vittoria di Heuristic Minimax in 6 mosse

Figura 5.3: report HeuristicMinimax vs GreedySearch partendo dai Paesi Bassi

Dai report è possibile notare come è presente una parte in comune: la partita inizia con i Paesi Bassi e GreedySearch opta la Germania. A questo punto, si assiste a comportamenti differenti da parte dei due algoritmi che giocano in casa:

- AlphaBetaPruning, dopo aver esplorato l'intero albero (e potato dove opportuno), sceglie il Lussemburgo in quanto, avendo il ruolo di *MAX* in questo game e trovandosi nello Stato Belgio, lo score x associato ad esso è superiore o uguale a quello associato a Francia e Germania (le altre possibili mosse legali);

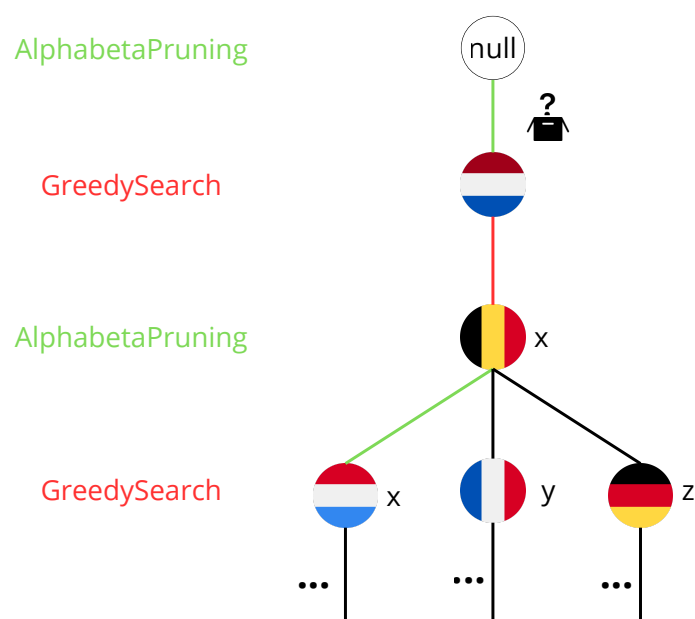


Figura 5.4: albero di gioco AlphaBetaPruning vs GreedySearch partendo dai Paesi Bassi

- HeuristicMinimax, invece, guarda in avanti di 10 mosse e si accorge che andando in Lussemburgo, arriva in un nodo intermedio (score 0); andando in Francia, arriva in un altro nodo intermedio (score 0); infine, andando in Germania, c'è la possibilità di arrivare ad un nodo terminale e la sceglie. Questo perché lo score x associato è sicuramente superiore a 0 in quanto svolge il ruolo di MAX. Andare in Germania non era la scelta migliore in assoluto, altrimenti AlphaBetaPruning l'avrebbe effettuata, ma siccome l'euristica che usa la Greedysearch non è perfetta, ha condotto ad una vittoria di HeuristicMinimax.

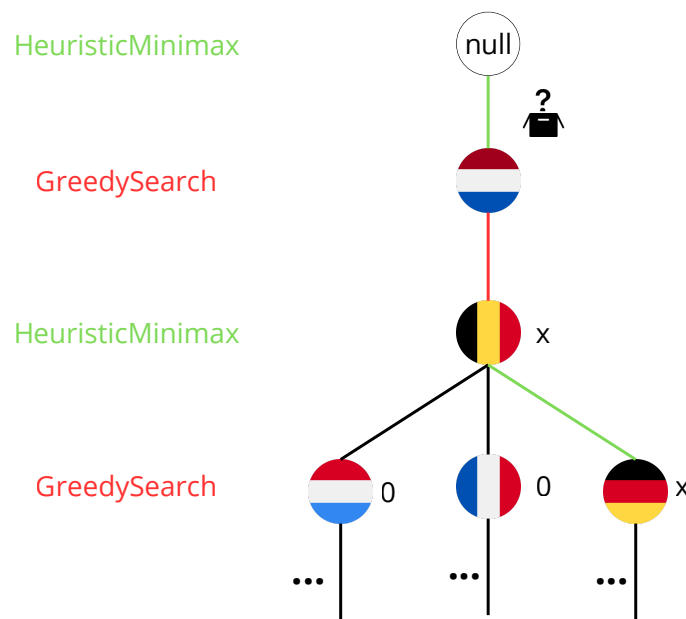


Figura 5.5: albero di gioco HeuristicMinimax vs GreedySearch partendo dai Paesi Bassi

Quanto ai tempi medi per eseguire una mossa, la pratica conferma ciò che già si sapeva dalla teoria:

- StandardMinimax è il più lento della famiglia, con un tempo medio per effettuare una mossa di circa 0,8 secondi;
- AlphaBetaPruning risulta essere circa **90 volte** più veloce rispetto alla sua versione non ottimizzata;
- HeuristicMinimax, invece, è circa **10 volte** più rapido di AlphaBetaPruning e addirittura **900 volte** più veloce di StandardMinimax.

AlphaZero

AlphaZero si è rivelato essere il migliore, per punti fatti e game vinti, degli algoritmi non appartenenti alla famiglia Minimax. Purtroppo, il dato relativo al tempo medio impiegato per individuare la prossima mossa da eseguire (circa 144 *ms*), va preso con le pinze in quanto il codice che lo riguarda è stato scritto in Python mentre tutto il resto dell'applicazione è in Kotlin. Infatti, si tratta di due approcci totalmente differenti per quanto riguarda la modalità di compilazione (infatti il codice Kotlin viene compilato, mentre quello in Python viene interpretato). Inoltre, il prodotto tra matrici è una delle operazioni più CPU intensive che esistano anche se nel calcolo del tempo impiegato non viene incluso l'overhead necessario per trasferire i dati dal server FastAPI (in Python) al client OkHTTP (in Kotlin).

Altra considerazione che è possibile fare su questo algoritmo è che sia contro GreedySearch che contro MonteCarloTreeSearch, indipendentemente da chi fosse il primo a giocare, è riuscito a portare a casa almeno 20 game sui 34 disponibili (circa il 59%). Le maggiori difficoltà sono state riscontrate contro la famiglia Minimax, soprattutto quando era il primo a muovere, riuscendo a perdere tutti i match a zero. Per le limitate risorse di calcolo disponibili e per l'addestramento ricevuto, le prestazioni offerte sono da ritenersi più che accettabili.

MonteCarloTreeSearch

Un gradino più sotto ad AlphaZero, c'è MonteCarloTreeSearch in grado di battere soltanto GreedySearch e con un tempo medio richiesto per individuare la prossima mossa da eseguire di soli 3,8 *ms*. Proprio come AlphaZero ha perso tutti i match a zero quando giocava in "casa" contro gli algoritmi della famiglia Minimax. Probabilmente sono da rivedere non solo le iterazioni (1500 sono troppo poche), ma anche i criteri usati nelle quattro fasi di cui si compone.

GreedySearch

Nonostante occupi l'ultimo posto in classifica, ci sono diverse ragioni per le quali essere soddisfatti di questa implementazione dell'algoritmo:

- il tempo medio è irrisorio se paragonato a quello degli avversari (circa 5 μ s);

- è riuscito a strappare il maggior numero di game agli algoritmi della famiglia Minimax, ben 14, rispetto agli 11 di AlphaZero e ai 7 di MonteCarloTreeSearch, ragion per cui l'euristica utilizzata ha una certa affidabilità.

5.6.2 Campionato 2

Non hanno preso parte a questo campionato né StandardMinimax, né la sua versione ottimizzata AlphaBetaPruning in quanto, in fase di testing, il tempo necessario per calcolare una sola mossa superava abbondantemente i 60 secondi vista l'enormità dell'albero di gioco, ragion per cui si è scelto di escluderli dalla competizione. Di seguito alcune considerazioni sull'esito del campionato 2, anche dal punto di vista comparativo, ove possibile, con il campionato 1.

HeuristicMinimax

Per HeuristicMinimax, la classifica parla da sola: sei partite, altrettante vittorie. La cosa impressionante sono i game persi: soltanto 56 su 1176 totali (circa il 4,76%) e tutti per opera dello stesso avversario: GreedySearch. A parità di profondità del taglio utilizzata, risulta essere aumentato il tempo medio per individuare la prossima mossa da effettuare: si è passati dai circa 0,9 *ms* del campionato 1 a più di 120 *ms* nel campionato 2. Incremento del 13.000% circa, dovuto presumibilmente all'aumento del fattore di ramificazione dell'albero di gioco considerato. Resta comunque un tempo più che soddisfacente per usi concreti.

GreedySearch

GreedySearch può essere considerato la sorpresa di questo secondo torneo per tutta una serie di motivi:

- è stato l'unico algoritmo a strappare qualche game a HeuristicMinimax, a testimonianza, ancora una volta, della bontà dell'euristica scelta;
- ha vinto il 65% dei game disputati, anche forse per demerito degli avversari;
- se si confrontano i tempi medi, quello di GreedySearch è almeno diecimila volte inferiore a qualsiasi altro algoritmo coinvolto nel torneo;

Per le motivazioni sopra elencate, GreedySearch è da considerarsi il giusto compromesso tra qualità delle mosse e tempo impiegato per calcolarle in questo secondo campionato.

AlphaZero

I risultati scadenti ottenuti da AlphaZero sono principalmente imputabili alla inadeguatezza degli strumenti usati per il suo addestramento. Infatti, la poca potenza di calcolo disponibile, ha costretto a impostare dei parametri distanti anni luce da quelli che vengono usati nella realtà per addestrare un algoritmo del genere. Basti pensare che per addestrare DeepMind sono state effettuate nove 44 milioni di self-play [10], contro le 200 per ciclo di questa implementazione. Anche le sole 30 iterazioni della Monte Carlo tree search sono da ritenersi più che insufficienti. Eppure, con questo scarso addestramento si è riusciti a battere per ben due volte MonteCarloTreeSearch. Anche il tempo di calcolo della mossa successiva è estremamente proibitivo: 2,5 s, cosa che lo rende inutilizzabile in implementazioni concrete.

MonteCarloTreeSearch

Fanalino di cosa, nonché vincitore indiscusso del celeberrimo "cucchiaino di legno"⁴, è risultato essere MonteCarloTreeSearch. Con tutta probabilità, le cause del fallimento di questo algoritmo, sia qui che nel campionato precedente, sono da imputarsi ai criteri scelti nelle varie fasi in cui esso si articola.

⁴Noto ai più come *Wooden Spoon*, trattasi di un riconoscimento goliardico del Sei Nazioni, assegnato a chi occupa l'ultimo posto

Conclusioni e sviluppi futuri

6.1 CONCLUSIONI

Le conclusioni che è possibile trarre dall'analisi empirica condotta nel capitolo precedente, sono essenzialmente le seguenti:

- in BorderGuessr, chi muove per secondo ha un leggero vantaggio rispetto a chi muove per primo, o meglio, con le due mappe considerate per l'analisi empirica questo accade. Infatti quando si sono affrontati MinimaxStandard e AlfaBetaPruning (letteralmente lo stesso algoritmo), il risultato è sempre stato a favore di chi muoveva per secondo (con un punteggio finale di 2-32);
- le differenze di performance tra Minimax standard, Alfa-Beta e Minimax euristico sono evidenti: Alfa-Beta risulta essere più veloce di circa 90 volte rispetto al Minimax classico (probabilmente anche l'ordinamento delle mosse è risultato essere abbastanza favorevole). La cosa più sorprendente è però che il Minimax euristico arriva ad essere 900 volte più rapido di quest'ultimo con una profondità di taglio pari a 10 (che non è per niente bassa se si pensa che i Gran Maestri negli scacchi sono in grado di guardare circa 10-15 mosse in avanti¹).

¹Fonte: https://www.reddit.com/r/chessbeginners/comments/1dfdfd/zh/how_many_moves_into_the_future_should_you/

- il Minimax euristico risulta essere l'opzione migliore se la macchina che si possiede ha poca potenza di calcolo; non solo, grazie alla sua subottimalità, riesce a comportarsi meglio contro avversari imperfetti, rispetto a quanto farebbe un algoritmo ottimale come Minimax standard. Tutto questo a conferma della bontà di quanto proposto negli anni '50 da Claude E. Shannon [7], relativamente al fatto che, fissando una profondità massima di ricerca, era possibile ottenere comunque buoni risultati;
- gli algoritmi Greedy non sono da sottovalutare, poiché una buona euristica può essere determinante, anche contro algoritmi nettamente più forti dal punto di vista teorico;
- l'analisi empirica conferma che non esiste un algoritmo "universale" migliore; la strategia ottimale dipende dal contesto, dalla potenza di calcolo e dalle caratteristiche dell'avversario;

Invece, per quanto riguarda la versione mobile di BorderGuessr, con una certa dose di ottimismo (non avendo condotto test ad hoc su campioni statisticamente rilevanti), si può constatare, sulla base di considerazioni personali, che:

- l'applicativo riesce a combinare gioco e apprendimento in quanto obbliga il giocatore a ragionare sui confini politici attuali e sulla geografia globale, favorendo la memoria spaziale e la consapevolezza geografica;
- l'elemento competitivo aumenta il coinvolgimento perché sfidare algoritmi di intelligenza artificiale diversi permette di creare sfide variegata e imprevedibili, mantenendo alto l'interesse del giocatore;
- la breve durata delle partite rende l'applicativo un ottimo passatempo durante i tempi morti o momenti di relax.

6.2 SVILUPPI FUTURI

Il ruolo che il lavoro di analisi empirica svolto mira a ricoprire è quello di punto di partenza per condurre ulteriori analisi di questo tipo, magari con una o più delle seguenti modifiche:

- introdurre varianti degli algoritmi studiati, definendo nuove euristiche (ad esempio, scegliere lo Stato meno popoloso tra quelli disponibili; quello meno densamente abitato; quello con meno chilometri di confine...) e combinandole tra loro; cambiando gli iperparametri relativi all'addestramento di AlphaZero, magari con valori più vicini a quelli usati nella pratica; apportare ulteriori miglioramenti al Minimax, evitando la navigazione completa dell'albero di gioco e/o sfruttando il calcolo parallelo; definire più versioni dell'algoritmo MCTS;
- condurre le analisi su dispositivi con maggiore potenza di calcolo, preferibilmente con GPU dedicata e non integrata;
- considerare come parametro valutativo anche il numero medio di mosse per game, oltre al tempo medio per individuare una mossa;
- tentare nuovi approcci analitici, non necessariamente basati su round-robin;

Inoltre, occorre necessariamente sottolineare che la versione prodotta dell'applicativo mobile è molto «rudimentale». Per questo motivo, i suggerimenti per poterla migliorare sono i più disparati. Per citarne alcuni:

- aggiungere una sorta di indizi con informazioni peculiari di uno o più paesi ritenuti essere mosse valide, così da arricchire le conoscenze socio-culturali dell'utilizzatore;
- sulla stessa scia d'onda del punto precedente, sembra più che sensato prevedere alcuni tentativi prima notificare la sconfitta al giocatore umano;
- aggiungere più mappe, nonché la possibilità di crearne di personali escludendo paesi o aggiungendone altri;
- prevedere una modalità multi-giocatore, dove a sfidarsi non sono un uomo e la macchina, bensì due o più individui;
- estendere il concetto di BorderGuessr, concepito quindi non solo per gli Stati, ma anche per testare le proprie conoscenze sulle regioni d'Italia o d'Europa, sui comuni di una provincia, sulle strade di una città...;
- assegnare dei punti per ogni vittoria conseguita e tenerne traccia in una classifica locale/mondiale;

- prevedere più lingue di gioco, oltre al semplice italiano, così da permettere l'apprendimento linguistico oltre a quello prettamente geografico.

Bibliografia

- [1] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020. (Citato alle pagine 6, 7, 11, 22 e 23)
- [2] T. H. Kjeldsen, "John von neumann's conception of the minimax theorem: A journey through different mathematical contexts," *Archive for History of Exact Sciences*, vol. 56, pp. 39–68, 2001. (Citato a pagina 9)
- [3] M. H. Albert, R. J. Nowakowski, and D. Wolfe, *Lessons in Play*, 2nd ed. CRC Press, 2019. (Citato a pagina 9)
- [4] D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press, 2023. (Citato a pagina 11)
- [5] S. Zhang, "A study on search techniques in the game-tree," in *Proceedings of the 2023 International Conference on Machine Learning and Automation*, 2023, pp. 253–258. (Citato alle pagine 13, 19 e 23)
- [6] P. Mendoza. (2022) Alpha-beta pruning algorithm: The intelligence behind strategy games. [Online]. Available: https://www.researchgate.net/publication/360872512_Alpha-Beta_Pruning_Algorithm_The_Intelligence_Behind_Strategy_Games (Citato alle pagine 14 e 18)

- [7] C. E. Shannon, "Xxii. programming a computer for playing chess," *Philosophical Magazine*, vol. 41, 1950. (Citato alle pagine 20 e 76)
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. The MIT Press, 2020. (Citato a pagina 21)
- [9] G. Jain, A. Kumar, and S. A. Bhat, "Recent developments of game theory and reinforcement learning approaches: A systematic review," *IEEE Access*, 2024. (Citato a pagina 21)
- [10] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. (2017) Mastering chess and shogi by self-play with a general reinforcement learning algorithm. [Online]. Available: <https://arxiv.org/pdf/1712.01815> (Citato alle pagine 24 e 74)
- [11] D. A. McFarlane, "Critical review of geographic illiteracy in the united states: A multifaceted analysis," *Journal of Geography, Environment and Earth Science International*, 2024. (Citato a pagina 25)
- [12] RoperASW, "National geographic — roper 2002 global geographic literacy survey," National Geographic Education Foundation, Tech. Rep., 2002. (Citato a pagina 26)
- [13] Álvaro Francisco Morote and M. Hernández-Hernández, "Gamification in geography. use, appropriateness and proposals according to university students in spain," *European Journal of Geography*, 2024. (Citato a pagina 27)
- [14] G. E. Moore, "Cramming more components onto integrated circuits," *Electronics*, vol. 38, 1965. (Citato a pagina 56)